COLLECTION TOTAL — L'ART DE RÉALISER UN PRODUIT LOGICIEL

Fondations — Livre 2 sur 14

Le SDLC ToTAL

Le cycle vivant de la maîtrise logicielle

SENSE · SPECIFY · PLAN · FORGE · DELIVERY · CARE

Michel Bélanger

Crédits visuels : Soléane Bélanger

Septembre 2026

© 2026 Michel Bélanger. Tous droits réservés. Toute reproduction partielle ou totale est interdite sans l'autorisation écrite de l'auteur. La collection ToTAL et ses composantes sont des œuvres originales protégées par le droit d'auteur.

« Un cycle sans apprentissage n'est qu'une répétition déguisée en progrès. »

— ToTAL, principe fondateur

À toutes les équipes qui ont cherché, un jour, à faire un peu mieux qu'hier.

Table des matières

Introduction — Le cycle comme discipline

Pourquoi un cycle doit devenir une discipline de maîtrise

Le problème fondamental : les cycles qui tournent sans apprendre

La promesse du SDLC ToTAL

Comment lire ce livre

Partie 1 — La philosophie du cycle ToTAL

Chapitre 1 — Qu'est-ce qu'un cycle de développement digne de ce nom ?

Chapitre 2 — Les six capacités du SDLC ToTAL : vue d'ensemble

Chapitre 3 — Le cycle comme système apprenant

Partie 2 — SENSE : Définir, Qualifier, Mesurer et Apprendre

Chapitre 4 — La première capacité structurante

Chapitre 5 — Outils et pratiques de SENSE

Chapitre 6 — SENSE et l'IA : clarifier avant d'amplifier

Partie 3 — SPECIFY : Les spécifications vivantes SI/ALORS

Chapitre 7 — Le langage naturel de la qualité testable

Chapitre 8 — Écrire des spécifications qui survivent au réel

Chapitre 9 — SPECIFY et la dette conceptuelle

Partie 4 — PLAN : Décider avec lucidité

Chapitre 10 — La planification comme acte de maîtrise

Chapitre 11 — Outils de planification ToTAL

Partie 5 — FORGE : Construire avec qualité intégrée

Chapitre 12 — Transformer la clarté en construction fiable

Chapitre 13 — Pratiques de FORGE

Chapitre 14 — Anti-patterns de FORGE

Partie 6 — DELIVERY et CARE : Livrer et apprendre

Chapitre 15 — DELIVERY : la livraison comme acte de confiance

Chapitre 16 — CARE : transformer la livraison en boucle d'apprentissage

Chapitre 17 — Boucler le cycle : de CARE à SENSE

Partie 7 — Le SDLC ToTAL dans votre contexte

Chapitre 18 — ToTAL et les cadres existants : Agile, SAFe, DevOps

Chapitre 19 — Démarrer avec le SDLC ToTAL : premières semaines

Épilogue — Le cycle vivant

Annexes

Annexe A — Gabarit de plan de qualité par phase SDLC ToTAL

Annexe B — Référentiel SI/ALORS : 25 exemples commentés

Annexe C — Grille d'auto-évaluation SDLC ToTAL

Annexe D — Glossaire ToTAL SDLC

INTRODUCTION

Le cycle comme discipline

Pourquoi un cycle de développement logiciel doit devenir une discipline de maîtrise

Il existe, dans chaque organisation qui produit du logiciel, une vérité que l'on n'aime pas formuler clairement : la plupart des cycles de développement sont des routines déguisées en méthodes. On les appelle « processus agiles », « pipelines DevOps », « cadres de livraison continue » — et ces noms sont souvent mérités, au sens technique du terme. Mais derrière les cérémonies, les sprints et les tableaux Kanban, on retrouve fréquemment la même réalité fondamentale : une séquence d'actions qui se répète, sans que l'équipe comprenne vraiment pourquoi elle obtient les résultats qu'elle obtient. Le cycle tourne. La roue tourne. Mais rien n'apprend.

Ce livre est né d'une conviction profonde et d'une observation répétée sur le terrain : un cycle de développement logiciel qui ne devient pas une discipline de maîtrise finit toujours par trahir ceux qui lui font confiance. Il trahit les utilisateurs qui espèrent un

produit qui répond à leurs vrais besoins. Il trahit les équipes qui investissent leur énergie et leur intelligence dans un travail qui disparaît sous l'accumulation des correctifs. Il trahit les organisations qui ont cru que la vitesse de livraison suffisait à garantir la valeur livrée.

La maîtrise, dans le sens que ToTAL lui donne, n'est pas un état d'arrivée. Ce n'est pas un certificat qu'on accroche au mur après avoir complété une formation. C'est une posture active, quotidienne, collective : celle d'une équipe qui observe ce qu'elle fait, qui comprend pourquoi elle le fait, qui mesure ce que cela produit, et qui apprend de l'écart entre ce qu'elle voulait produire et ce qu'elle a réellement produit. La maîtrise, c'est la capacité à transformer chaque cycle en preuve de compréhension — pas seulement en preuve d'exécution.

Le SDLC ToTAL — Software Development Life Cycle Total — est la réponse de la collection ToTAL à cette question fondamentale : comment concevoir un cycle de développement qui apprend ? La réponse ne passe pas par l'ajout d'une nouvelle cérémonie, d'un nouveau rôle ou d'un nouveau tableau de bord. Elle passe par une reconception de ce que chaque phase du cycle doit produire : non seulement un livrable, mais un apprentissage. Non seulement une étape franchie, mais une qualité démontrée.

Le problème fondamental : les cycles qui tournent sans apprendre

Posons le problème avec précision. Dans la grande majorité des organisations qui développent du logiciel, le cycle de développement souffre d'une pathologie que l'on pourrait appeler l'**exécution sans réflexion**. On planifie des sprints sans se demander si les backlog items sont réellement clairs. On code des fonctionnalités sans s'assurer que les spécifications reflètent un vrai besoin utilisateur. On livre des releases sans vérifier que ce qui a été livré correspond à ce qui avait été compris. Et on recommence. Sprint après sprint. Release après release. Cycle après cycle.

Le symptôme le plus visible de cette pathologie n'est pas le bug — même si les bugs sont nombreux. C'est le retravailler. L'équipe qui passe trente pour cent de son temps à corriger ce qu'elle a produit deux sprints auparavant. Le produit qui accumule les

fonctionnalités rarement utilisées parce que les besoins n'avaient pas été vraiment compris. La démonstration de fin de sprint où le Product Owner dit, pour la troisième fois consécutive : « Ce n'est pas tout à fait ce que je voulais dire. » Ce retravailler coûte des ressources. Il coûte de la confiance. Il coûte de l'élan collectif. Et surtout, il signale que le cycle ne sert pas à apprendre — il sert à produire, dans l'espoir que la production finira par être juste.

⚠️ Symptômes — Le cycle qui tourne sans apprendre

- Plus de trente pour cent du temps de sprint consacré à corriger le travail de sprints antérieurs.
- Les rétrospectives produisent les mêmes constats depuis quatre cycles ou plus.
- Le Product Owner découvre en démo ce que l'équipe a compris de son besoin.
- Les critères d'acceptation sont écrits après le développement, par l'équipe elle-même.
- L'équipe ne peut pas expliquer pourquoi la qualité de sa dernière livraison était ce qu'elle était.
- Les incidents de production reviennent sur des catégories identiques cycle après cycle.

Ce problème a des causes multiples, mais l'une d'elles est particulièrement insidieuse parce qu'elle se cache dans la structure même du cycle. La plupart des SDLC — qu'ils soient agiles, hybrides ou classiques — sont conçus comme des séquences d'étapes à franchir. L'accent est mis sur la transition entre les étapes (le passage de la conception au développement, du développement au test, du test à la livraison), pas sur ce que chaque étape doit avoir prouvé avant d'autoriser la suivante. En d'autres termes, le cycle mesure l'avancement, pas la qualité de l'apprentissage. Il valide que quelque chose a été fait, pas que quelque chose a été compris.

ToTAL renverse cette logique. Dans le SDLC ToTAL, chaque phase — SENSE, SPECIFY, PLAN, FORGE, DELIVERY, CARE — possède ce qu'on appellera une **preuve de qualité de phase** : un ensemble d'indicateurs, de livrables et de démonstrations qui confirment non seulement que la phase a été exécutée, mais que ce qu'elle devait permettre de comprendre a effectivement été compris. Sans cette preuve, la phase n'est pas terminée — elle est seulement arrêtée.

La promesse du SDLC ToTAL : relier chaque phase à une preuve de qualité

La promesse centrale du SDLC ToTAL tient en une phrase : **chaque phase du cycle produit un apprentissage démontrable, pas seulement un livrable consommable**. Cette distinction est cruciale. Un livrable consommable est quelque chose qu'on remet à la phase suivante : un document de besoins, un ensemble de user stories, un plan de sprint, un incrément logiciel, un rapport de test. Ces livrables sont nécessaires — mais ils ne sont pas suffisants. Ce qui compte, c'est ce que ce livrable *démontre* : que l'équipe a vraiment compris le besoin avant de le spécifier, qu'elle a vraiment clarifié avant de construire, qu'elle a vraiment appris avant de recommencer.

ToTAL est construit sur une chaîne causale que l'on retrouvera tout au long de ce livre : **Clarté → Qualité → Confiance**. La clarté — la compréhension partagée et précise de ce qu'on cherche à accomplir — est la condition préalable à la qualité. La qualité — la conformité vérifiable entre ce qui a été compris et ce qui a été produit — est la fondation de la confiance. La confiance — la capacité de l'équipe, des parties prenantes et des utilisateurs à compter sur ce que le cycle produit — est le résultat naturel d'un cycle sain. Cette chaîne n'est pas linéaire : elle est cyclique. La confiance accumulée au cours d'un cycle alimente la clarté du suivant. L'apprentissage de CARE enrichit le SENSE du prochain cycle. Le cycle devient vivant.

> **📌 À retenir — La chaîne ToTAL**
>
> **Clarté → Qualité → Confiance.** Cette chaîne gouverne chaque phase du SDLC

ToTAL. Toute dégradation de la clarté se répercute sur la qualité. Toute dégradation de la qualité érode la confiance. Et toute perte de confiance rend la clarté encore plus difficile à atteindre au prochain cycle. Maîtriser cette chaîne, c'est maîtriser le cycle.

Comment lire ce livre

Ce livre est le deuxième de la collection ToTAL. Il peut se lire de façon autonome — chaque partie et chaque chapitre sont conçus pour être accessibles sans avoir lu le Livre 1. Mais il prend tout son sens dans la continuité de la collection : les concepts de qualité holistique, de dette conceptuelle, et de lexique vivant introduits dans le Livre 1 sont ici mis en mouvement dans le cadre du cycle de développement.

Vous trouverez dans ce livre deux niveaux de lecture. Le premier est conceptuel : comprendre pourquoi le SDLC ToTAL est structuré comme il l'est, quels principes gouvernent chaque phase, et pourquoi ces principes importent. Le second est opérationnel : des ateliers guidés, des grilles d'évaluation, des anti-patterns détaillés, et des exemples concrets tirés de la mission fictive ORIGINALIUM, portée par l'équipe DREAM-TEAM. Cette équipe fictive accompagne tout le livre comme fil conducteur narratif : elle fait des erreurs instructives, découvre des solutions par le terrain, et illustre comment les concepts ToTAL s'appliquent dans la réalité d'une équipe sous pression.

Chaque chapitre se termine par une **Phrase repère** — une formulation condensée de l'idée centrale du chapitre, à mémoriser, à partager, à afficher. Ces phrases ne sont pas des slogans : elles sont des balises de navigation pour les moments où le cycle vacille et où l'équipe a besoin d'un point de retour. Lisez-les lentement. Revenez-y. Elles ont été écrites pour durer.

Phrase repère : *Un cycle de développement n'est pas une route que l'on suit — c'est une discipline que l'on pratique, jour après jour, phase après phase, preuve après preuve.*

Partie 1 — La philosophie du cycle ToTAL

Comprendre avant de méthoder · Dix à douze pages · Trois chapitres

---

« Ce n'est pas la méthode qui fait le cycle — c'est la compréhension de ce que le cycle doit produire. »

CHAPITRE 1

Qu'est-ce qu'un cycle de développement digne de ce nom ?

Le SDLC classique et ses limites : une comparaison honnête

L'histoire du développement logiciel est, en partie, une histoire de cycles. Depuis les années 1970, l'industrie a cherché à répondre à une question simple en apparence : dans quel ordre doit-on réaliser les activités nécessaires à la production d'un logiciel ? La réponse à cette question a généré des dizaines de modèles, de cadres et de méthodes. Certains ont eu une influence durable. D'autres ont disparu rapidement. Mais tous ont partagé la même ambition : donner à la production logicielle une structure qui permette de prédire, contrôler et améliorer les résultats.

Le modèle en cascade (Waterfall), introduit formellement par Winston Royce en 1970 — et souvent cité à contresens, puisque Royce lui-même reconnaissait ses limites — propose une progression linéaire : exigences, conception, implémentation, vérification, maintenance. L'attrait de ce modèle était sa clarté : chaque phase avait

un début, une fin et un livrable défini. Mais sa limite fondamentale était tout aussi claire : dans un monde où les besoins évoluent, où les technologies surprennent et où les utilisateurs découvrent ce qu'ils veulent en voyant ce que l'équipe produit, la linéarité devient une rigidité dangereuse. On découvrait en phase de vérification que la conception avait mal interprété les exigences. On découvrait en maintenance que les exigences avaient mal capturé le vrai besoin. Et il était alors trop tard, trop coûteux, trop douloureux pour recommencer.

L'Agile, formalisé en 2001 avec le Manifeste pour le développement agile de logiciels, est venu corriger précisément cette rigidité. En valorisant les individus et leurs interactions, les logiciels fonctionnels, la collaboration avec les clients, et la réponse au changement, l'Agile a introduit l'idée fondamentale que le cycle doit être court, itératif et adaptatif. Scrum, XP, Kanban et leurs variantes ont apporté des outils concrets pour mettre en œuvre cette idée. Et l'Agile a été un vrai progrès — un progrès réel, mesurable, important. Mais avec le temps, une nouvelle pathologie est apparue : l'agilité de façade. Des cycles courts qui ne livrent pas plus vite de la valeur réelle. Des rétrospectives qui ne produisent pas de vrais changements. Un backlog qui gonfle indéfiniment. Une vitesse mesurée en points d'histoire qui ne dit rien de la qualité de ce qui est produit.

DevOps, apparu dans les années 2010 comme mouvement culturel avant de devenir un ensemble de pratiques et d'outils, a apporté une réponse à une autre limite : la déconnexion entre les équipes de développement et les équipes d'opérations. En unissant Dev et Ops autour d'une chaîne d'intégration et de livraison continues (CI/CD), DevOps a considérablement réduit les délais entre l'écriture du code et sa mise en production. Mais DevOps est avant tout un mouvement d'automatisation et de vitesse. Il répond remarquablement bien à la question « comment livrer plus vite ? » — mais beaucoup moins bien à la question « comment s'assurer que ce qu'on livre est ce qui était vraiment voulu ? »

Modèle	Point fort principal	Limite principale	Ce que ToTAL y ajoute
Waterfall	Structure claire, documentation complète,	Rigidité face au changement, découverte tardive	Preuves de qualité de phase avant passage à la

Modèle	Point fort principal	Limite principale	Ce que ToTAL y ajoute
	prévisibilité en contexte stable	des erreurs de compréhension	suivante ; apprentissage intégré
Agile / Scrum	Itérativité, adaptabilité, implication client continue, livraison rapide	Qualité sacrifiée à la vitesse, backlog inflationnaire, cérémonies sans substance	Chaîne Clarté → Qualité → Confiance ; spécifications vivantes SI/ALORS
DevOps / CI-CD	Automatisation, livraison continue, réduction des délais Dev-Ops	Vitesse sans compréhension, amplification des erreurs de conception en production	SENSE avant automation ; CARE comme boucle d'apprentissage post-livraison
SAFe	Alignement à grande échelle, gouvernance, planification trimestrielle	Lourdeur, bureaucratie, perte d'agilité réelle dans les grandes organisations	SPECIFY comme pont entre stratégie SAFe et implémentation qualité
SDLC ToTAL	Qualité holistique, apprentissage par phase, discipline de maîtrise	Exige une maturité d'équipe et une volonté d'investir dans la clarté	— (C'est le modèle de référence de ce livre)

Ce que le SDLC ToTAL ajoute : la qualité holistique comme fil conducteur

Le SDLC ToTAL n'est pas un nouveau cadre concurrent de Scrum ou de SAFe. Il ne cherche pas à remplacer vos cérémonies agiles, votre pipeline CI/CD ou vos rituels de planification. Il cherche à leur donner une colonne vertébrale : la qualité holistique comme fil conducteur de chaque phase du cycle. Par qualité holistique, ToTAL entend une qualité qui se préoccupe autant de la clarté des intentions que de la conformité des livrables, autant de la qualité du processus de compréhension que de la qualité du produit livré, autant de la qualité des relations entre les membres de l'équipe que de la qualité du code dans le dépôt.

Cette vision holistique de la qualité est ce qui distingue fondamentalement ToTAL des approches de qualité traditionnelles, qui se concentrent presque exclusivement sur la phase de test et sur la conformité des livrables à des critères techniques. Dans ToTAL,

la qualité commence au moment où l'équipe essaie de comprendre un besoin — et elle ne s'arrête pas quand le code est en production. Elle continue dans la phase CARE, au travers des observations post-livraison, des métriques d'usage réel, et des apprentissages qui alimenteront le prochain cycle.

La chaîne clarté → qualité → confiance dans chaque phase du cycle

La chaîne Clarté → Qualité → Confiance n'est pas un slogan. C'est un modèle causal qui permet de comprendre, diagnostiquer et améliorer n'importe quelle phase du cycle. Voici comment cette chaîne se manifeste concrètement dans chacune des six phases du SDLC ToTAL.

Dans SENSE, la clarté signifie que l'équipe a compris le besoin dans sa vérité — pas dans sa formulation première, souvent trompeuse. La qualité, c'est que cette compréhension a été qualifiée, mesurée et documentée de façon à pouvoir être vérifiée. La confiance, c'est que les parties prenantes et l'équipe peuvent s'appuyer sur ce qui a été compris pour décider si l'on passe à SPECIFY.

Dans SPECIFY, la clarté signifie que chaque besoin compris a été traduit en une spécification précise, testable et traçable. La qualité, c'est que ces spécifications sont vivantes — elles reflètent le réel, elles évoluent avec le réel, et elles permettent de générer des tests. La confiance, c'est que tout le monde — développeurs, testeurs, Product Owner — comprend de la même façon ce que le logiciel doit faire.

Dans FORGE, la clarté signifie que les développeurs savent précisément ce qu'ils construisent et pourquoi. La qualité, c'est que ce qui est construit est conforme aux spécifications et qu'il est vérifié automatiquement à chaque intégration. La confiance, c'est que l'équipe peut livrer sans craindre d'avoir introduit des régressions cachées.

Anti-pattern : le cycle qui produit sans comprendre

Avant de passer au chapitre suivant, il est utile de nommer explicitement l'anti-pattern le plus courant du développement logiciel : le cycle qui produit sans comprendre. Cet anti-pattern se reconnaît à plusieurs signes. L'équipe passe

directement de la collecte des demandes à l'écriture du code, sans avoir vérifié que les demandes correspondent à de vrais besoins. Les user stories sont écrites pendant le sprint, pas avant. Les critères d'acceptation sont confirmés après les démos, pas avant les développements. La définition de « done » est une liste de tâches techniques, pas une preuve de qualité utilisateur.

> **🌱 Terrain — DREAM-TEAM, semaine 0**
>
> Avant de démarrer la mission ORIGINALIUM, l'équipe DREAM-TEAM — composée de sept personnes aux rôles variés — a passé sa première réunion de lancement à discuter des features à développer. La liste des features avait été préparée par le commanditaire, un directeur produit enthousiaste et pressé. Personne n'avait posé la question fondamentale : « Comment savons-nous que ces features répondent aux vrais besoins des utilisateurs ? » La réponse implicite de l'équipe était : « Parce que le commanditaire les a demandées. » C'était le début d'un cycle qui allait produire — beaucoup, vite — sans comprendre. Ce livre raconte, entre autres, comment DREAM-TEAM a appris à changer cette posture.

Phrase repère : *Un cycle digne de ce nom ne se mesure pas à ce qu'il a produit, mais à ce qu'il a compris en le produisant.*

CHAPITRE 2

Les six capacités du SDLC ToTAL : vue d'ensemble

Le SDLC ToTAL est organisé autour de six capacités fondamentales, chacune représentant une phase du cycle et une posture distincte que l'équipe adopte à ce moment-là. Le mot « capacité » est choisi délibérément. Il ne s'agit pas de phases au

sens mécanique du terme — des boîtes dans lesquelles on range des activités. Il s'agit de capacités organisationnelles et collectives : des aptitudes que l'équipe développe, renforce et mobilise à des moments précis du cycle. Une équipe qui maîtrise ces six capacités possède un cycle vivant. Une équipe qui exécute ces six phases sans les comprendre possède un processus mort.

SENSE : ressentir avant d'agir

SENSE est la capacité de comprendre le monde avant d'y agir. Dans le contexte du développement logiciel, « ressentir » ne signifie pas être passif ou contemplatif. Cela signifie déployer une attention active et structurée sur ce que les utilisateurs, les parties prenantes et le contexte d'usage révèlent sur le vrai besoin — avant de se précipiter vers la solution. SENSE est la phase la plus critique du cycle, parce que les erreurs qui y sont commises se propagent dans toutes les phases suivantes, amplifiées à chaque étape.

> 📌 **À retenir — SENSE en une phrase**
>
> **SENSE, c'est l'art de rendre le besoin visible avant que le problème ne devienne urgent.** L'urgence est l'ennemie de la compréhension. SENSE crée le temps et la méthode nécessaires pour comprendre juste — pas vite.

SPECIFY : nommer avant de construire

SPECIFY est la capacité de traduire ce qui a été compris dans SENSE en spécifications précises, testables et traçables. Le mot « nommer » est crucial : on ne peut construire que ce qu'on a su nommer. Les spécifications floues — « le système doit être performant », « l'interface doit être conviviale » — ne sont pas des spécifications. Ce sont des espoirs. SPECIFY, dans ToTAL, repose sur le format SI/ALORS : une syntaxe de spécification qui force la précision, la testabilité et la traçabilité en même temps. Spécifier, c'est aussi le moment où l'équipe découvre les incohérences, les contradictions et les ambiguïtés dans sa compréhension du besoin. SPECIFY est donc aussi un outil de diagnostic de SENSE.

PLAN : décider avant d'exécuter

PLAN est la capacité de transformer ce qui a été spécifié en un plan d'exécution qui intègre explicitement la qualité. La planification est souvent réduite à une estimation d'effort et à une allocation de ressources. ToTAL la reconçoit comme un acte de maîtrise : un moment où l'équipe décide consciemment de ce qu'elle va construire, dans quel ordre, avec quels critères de qualité, et avec quelle gestion des risques. PLAN protège l'équipe contre deux pathologies symétriques : le sur-engagement (promettre plus que ce qu'on peut livrer avec qualité) et le sous-engagement (se protéger au détriment de la valeur livrée).

FORGE : construire avant de diffuser

FORGE est la capacité de construire avec qualité intégrée — pas vérifiée après coup. Dans FORGE, la qualité n'est pas une étape de contrôle en fin de phase. Elle est tissée dans chaque moment du développement : dans les tests écrits avant le code, dans les revues de code comme dialogues qualité, dans l'intégration continue comme signal permanent de santé du système. FORGE mobilise toutes les capacités techniques de l'équipe, mais elle les mobilise au service d'une intention claire héritée de SENSE et SPECIFY.

DELIVERY : livrer avec confiance

DELIVERY est la capacité de mettre ce qui a été construit entre les mains de ceux pour qui cela a été construit — avec la confiance que ce qui est livré est ce qui était attendu, dans l'état de qualité qui a été promis. DELIVERY n'est pas une simple opération technique de déploiement. C'est un acte social et professionnel : une déclaration collective que « ce que nous livrons est digne de confiance ». Cette déclaration n'est possible que si SENSE, SPECIFY et FORGE ont correctement joué leurs rôles. DELIVERY est le moment où la chaîne Clarté → Qualité → Confiance rend son verdict.

CARE : apprendre après la livraison

CARE est sans doute la capacité la plus négligée — et la plus transformatrice. Dans la plupart des organisations, la livraison marque la fin du cycle. Dans ToTAL, elle marque le début de CARE : une phase d'observation active, de mesure, de compréhension et d'apprentissage à partir de ce que la mise en usage réel révèle sur la qualité de ce qui a été livré. CARE répond à la question que la plupart des cycles n'osent pas poser : « Ce que nous avons livré a-t-il vraiment produit la valeur que nous espérions ? Et qu'est-ce que cela nous apprend pour la suite ? »

Capacité	Question centrale	Preuve de qualité de phase	Erreur fatale
SENSE	Avons-nous compris le vrai besoin ?	Grille de qualification des besoins complétée et validée	Passer à SPECIFY sans avoir qualifié les besoins
SPECIFY	Avons-nous traduit le besoin en critères testables ?	Spécifications SI/ALORS traçables et validées par les parties prenantes	Écrire des user stories sans critères d'acceptation testables
PLAN	Avons-nous décidé avec lucidité ?	Plan intégrant risques qualité, responsabilités et critères de sortie	Planifier l'effort sans planifier la qualité
FORGE	Avons-nous construit ce que nous avions spécifié ?	Tests automatisés passants, revues de code complétées, intégration continue verte	Construire vite sans tester en continu
DELIVERY	Avons-nous livré avec les preuves de qualité suffisantes ?	Critères de sortie satisfaits, contrat qualité de livraison signé	Livrer sous pression sans vérifier les critères de sortie
CARE	Avons-nous appris de ce que nous avons livré ?	Métriques post-livraison analysées, apprentissages documentés, SENSE suivant alimenté	Fermer le cycle sans observer l'usage réel

Phrase repère : *Les six capacités du SDLC ToTAL ne sont pas six étapes à cocher — ce sont six postures à adopter, six façons d'être une équipe qui apprend.*

CHAPITRE 3

Le cycle comme système apprenant

Pourquoi chaque phase doit produire un apprentissage, pas seulement un livrable

Il existe une différence fondamentale entre un système d'exécution et un système apprenant. Un système d'exécution est optimisé pour produire des sorties à partir d'entrées, selon un processus défini. Il est efficace tant que les entrées et les contextes restent stables et que le processus a été correctement conçu. Un système apprenant, en revanche, est optimisé pour améliorer sa propre capacité à produire de meilleures sorties en fonction de ce qu'il observe, mesure et comprend au fil du temps. Le système apprenant n'est pas seulement efficace dans des conditions stables — il devient plus efficace à mesure qu'il accumule de l'expérience et en tire des enseignements.

Le SDLC ToTAL est conçu comme un système apprenant. Cela signifie que chaque phase du cycle doit produire deux types de sorties : des **livrables** (ce qui est transmis à la phase suivante) et des **apprentissages** (ce que l'équipe a découvert sur elle-même, sur le besoin, sur le produit, sur le processus, et sur le contexte). Les apprentissages ne s'évaporent pas à la fin de la phase — ils sont documentés, analysés et intégrés dans la pratique de l'équipe et dans le SENSE du cycle suivant.

Cette distinction entre livrables et apprentissages peut sembler subtile, mais ses implications pratiques sont profondes. Une équipe qui ne produit que des livrables répète les mêmes erreurs de cycle en cycle, parce qu'elle n'a pas de mémoire organisationnelle. Une équipe qui produit aussi des apprentissages accumule une intelligence collective qui lui permet d'améliorer progressivement non seulement la qualité de ses produits, mais la qualité de son cycle lui-même.

La boucle PDCA dans le SDLC ToTAL

La boucle PDCA — Plan, Do, Check, Act — est l'un des outils les plus puissants de l'amélioration continue, introduit par William Edwards Deming et popularisé dans le contexte de la gestion de la qualité industrielle. Dans le SDLC ToTAL, la boucle PDCA n'est pas une méthodologie externe qu'on applique au cycle — elle est intégrée structurellement dans le cycle lui-même. Chacune des six capacités peut être vue comme une application de la boucle PDCA à son propre domaine.

Dans SENSE, le Plan correspond à la définition du protocole d'élicitation des besoins. Le Do est l'exécution des ateliers, entretiens et observations. Le Check est la qualification des besoins recueillis contre les douze critères de la grille ToTAL. L'Act est la décision de passer à SPECIFY (si les besoins sont qualifiés) ou de retourner à l'élicitation (si des zones d'ombre subsistent). Cette boucle PDCA dans SENSE — une micro-boucle dans la grande boucle du cycle — est ce qui garantit que SENSE ne se contente pas de recueillir des demandes, mais qu'il produit de la compréhension vérifiée.

🔭 **Repères — La boucle PDCA dans le SDLC ToTAL**

Phase SDLC
Plan
Do
Check
Act
SENSE
Protocole d'élicitation
Ateliers, entretiens
Grille de qualification

Passer à SPECIFY ou boucler

SPECIFY

Choix des spécifications à rédiger

Rédaction SI/ALORS

Revue de traçabilité

Valider ou réviser les specs

PLAN

Canevas de planification

Estimation, priorisation

Revue des risques qualité

Ajuster le plan ou le valider

FORGE

Architecture technique

TDD, code, intégration

Tests automatisés, revues

Corriger ou livrer à DELIVERY

DELIVERY

Critères de sortie

Déploiement

Validation des critères

Confirmer ou retenir la livraison

CARE

Plan de mesure post-livraison

Observation, métriques

Analyse des apprentissages

Alimenter le SENSE suivant

Indicateurs de santé du cycle : métriques de qualité par phase

Un cycle apprenant a besoin d'indicateurs de santé — des métriques qui permettent à l'équipe de savoir si chaque phase produit les apprentissages et la qualité attendus. Ces indicateurs ne sont pas des métriques de reporting destinées au management : ce sont des signaux opérationnels que l'équipe utilise elle-même pour ajuster son comportement en temps réel. ToTAL propose un ensemble minimal d'indicateurs de santé par phase, que chaque équipe peut enrichir en fonction de son contexte.

Phase	Indicateur de santé clé	Valeur cible	Signal d'alerte
SENSE	Taux de qualification des besoins (% de besoins passant les 12 critères)	> 80%	< 60% : retour en élicitation obligatoire
SPECIFY	Taux de couverture spécification → test (% de specs avec au moins un test associé)	100%	< 90% : spécifications incomplètes
PLAN	Taux de risques qualité identifiés avec mitigation documentée	100%	< 75% : plan insuffisant
FORGE	Couverture de tests automatisés (% de code couvert)	> 80%	< 60% : dette technique croissante
DELIVERY	Taux de critères de	100%	< 100% : livraison

Phase	Indicateur de santé clé	Valeur cible	Signal d'alerte
	sortie satisfaits au moment de la livraison		non autorisée sans décision consciente documentée
CARE	% d'apprentissages post-livraison intégrés au prochain SENSE	> 70%	< 40% : boucle d'apprentissage brisée

Ces indicateurs ne sont pas des objectifs rigides à atteindre sous peine de sanction. Ce sont des boussoles. Une équipe dont le taux de qualification des besoins est à cinquante pour cent n'a pas « échoué » — elle a découvert quelque chose d'important sur la maturité de son processus SENSE, et elle peut maintenant décider comment s'améliorer. La mesure n'est pas là pour punir : elle est là pour rendre visible ce qui est invisible, et pour nommer ce qui ne serait autrement qu'une vague insatisfaction.

❓ Questions d'équipe — Le cycle comme système apprenant

- Pouvons-nous nommer trois choses concrètes que nous avons apprises lors de notre dernier cycle ?
- Comment ces apprentissages ont-ils modifié notre façon d'aborder le cycle en cours ?
- Avons-nous des métriques de qualité pour chaque phase de notre cycle actuel ?
- Quand nos rétrospectives produisent-elles des changements réels plutôt que des constats récurrents ?
- Qui est responsable de s'assurer que les apprentissages d'un cycle alimentent le suivant ?

Phrase repère : *Un cycle sans mémoire est condamné à recommencer ses erreurs avec l'enthousiasme du débutant et l'expérience du vétéran — mais sans jamais tirer avantage de l'une ni de l'autre.*

Partie 2 — SENSE : Définir, Qualifier, Mesurer et Apprendre

Ressentir avant d'agir · Douze à quinze pages · Trois chapitres

---

« Le besoin mal compris est le péché originel du logiciel. Tout ce qui vient après ne fait qu'en amplifier les conséquences. »

CHAPITRE 4

La première capacité structurante

Pourquoi SENSE précède tout : rendre la qualité visible avant le problème

SENSE est la première capacité du SDLC ToTAL, et elle mérite ce rang de priorité. Dans les cycles de développement traditionnels, on commence par recueillir des exigences — une activité souvent confondue avec SENSE, mais qui en est fondamentalement différente. Recueillir des exigences, c'est écrire ce que les parties prenantes demandent. SENSer, c'est comprendre ce dont les utilisateurs ont vraiment besoin — ce qui n'est pas toujours la même chose, et qui est rarement identique à ce que les commanditaires demandent.

Cette différence entre la demande formulée et le besoin réel est au cœur de l'utilité de SENSE. Considérons un exemple classique : un commanditaire demande « un tableau de bord avec plus de graphiques ». La demande est claire. Mais le besoin derrière cette demande est peut-être « comprendre rapidement si nos objectifs mensuels sont atteints ». Ces deux formulations conduisent à des solutions très différentes. La première conduit à ajouter des graphiques. La seconde conduit peut-être à repenser l'architecture complète du tableau de bord, ou à introduire un mécanisme d'alerte proactive qui remplace les graphiques. SENSE, c'est l'art de remonter de la demande au besoin — et de faire cette remontée de façon rigoureuse, vérifiable et partagée.

La qualité dans SENSE ne s'exprime pas en lignes de code ou en pages de documentation. Elle s'exprime dans la précision de la compréhension. Une équipe qui entre dans SPECIFY avec une compréhension claire, qualifiée et partagée de ce qu'elle cherche à accomplir a une probabilité bien supérieure de produire des spécifications justes — et donc un logiciel juste. Une équipe qui entre dans SPECIFY avec des besoins flous, incomplets ou contradictoires va fabriquer de la dette conceptuelle dès le début du cycle — une dette qui se remboursera, douloureusement, en correctifs, en retravailler et en perte de confiance.

SENSE dans la mission DREAM-TEAM : la semaine 1 de la mission ORIGINALIUM

🌱 Terrain — DREAM-TEAM, Semaine 1 de la mission ORIGINALIUM

La mission ORIGINALIUM avait pour objectif de livrer en six mois une plateforme de gestion de contenu éducatif pour un réseau de vingt-deux écoles primaires. L'équipe DREAM-TEAM — Alicia (Product Owner), Benoit (Tech Lead), Camille (UX Designer), David et Élise (Développeurs séniors), Fatima (QA Lead) et Guillaume (Scrum Master) — a démarré avec une conviction partagée : « On a déjà fait ce type de projet, on sait ce qu'il faut construire. »

La semaine 1 aurait dû être consacrée à SENSE. Elle a été consacrée à l'estimation. Au lieu d'organiser des ateliers avec les enseignants et les

directeurs d'école, DREAM-TEAM a produit un backlog de quatre-vingt-sept user stories basé sur les documents de requirements fournis par le commanditaire — des documents rédigés par un consultant externe qui n'avait interviewé que trois directeurs d'école sur vingt-deux. Six semaines plus tard, lors de la première démo, la principale enseignante-coordinatrice présente dans la salle a résumé la situation avec une politesse dévastatrice : « C'est très beau, mais ça ne ressemble pas du tout à ce que nous faisons en classe. »

DREAM-TEAM avait construit vite, bien, et à côté. SENSE avait été sauté. Le prix à payer : trois semaines de révision complète du backlog, deux sprints perdus, et une perte de confiance du commanditaire qui allait prendre quatre cycles à reconstruire.

Les quatre gestes de SENSE : écouter, nommer, qualifier, mesurer

SENSE n'est pas une activité unique — c'est une séquence de quatre gestes interdépendants, chacun contribuant à transformer les signaux bruts du terrain en compréhension qualifiée et mesurable.

Le premier geste est **écouter**. Écouter, dans SENSE, signifie créer les conditions dans lesquelles les vrais besoins peuvent émerger. Cela passe par des ateliers d'élicitation structurés, des entretiens individuels avec les utilisateurs finaux (pas seulement avec les commanditaires), des observations directes du travail réel (pas seulement des réunions de présentation), et une attention active aux contradictions, aux silences et aux formulations hésitantes qui révèlent souvent les besoins les plus importants. L'équipe qui écoute vraiment dans SENSE découvre fréquemment que les besoins les plus urgents n'étaient pas dans le document de requirements initial.

Le deuxième geste est **nommer**. Nommer, c'est transformer ce qui a été entendu en formulations précises, partagées et consensuelles. Le nommage dans SENSE n'est pas encore de la spécification — c'est le travail préalable qui rend la spécification possible. Il s'agit de construire un lexique commun, d'éliminer les ambiguïtés de

vocabulaire, et de s'assurer que quand deux personnes utilisent le même mot, elles parlent bien de la même réalité. Ce travail de nommage alimente directement le Lexique Vivant ToTAL.

Le troisième geste est **qualifier**. Qualifier un besoin, c'est vérifier que ce besoin répond à un ensemble de critères qui le rendent prêt à être spécifié. La grille de qualification des besoins ToTAL (voir Chapitre 5) propose douze critères organisés en quatre dimensions : Pertinence, Précision, Testabilité et Traçabilité. Un besoin qui ne satisfait pas ces critères n'est pas prêt pour SPECIFY — il doit retourner dans le cycle d'écoute et de nommage.

Le quatrième geste est **mesurer**. Mesurer dans SENSE signifie établir des indicateurs quantitatifs qui permettront de vérifier, dans les phases suivantes et après la livraison, que le besoin a bien été satisfait. Un besoin sans indicateur de mesure est un pari — on ne saura jamais si ce qui a été livré répond vraiment à ce qui était demandé. Les indicateurs de SENSE alimentent directement les critères d'acceptation de SPECIFY et les métriques post-livraison de CARE.

🚦 Signal — Les quatre gestes de SENSE

Écouter : Ateliers d'élicitation, entretiens utilisateurs, observation terrain
Nommer : Lexique commun, élimination des ambiguïtés, formulations partagées
Qualifier : Grille des douze critères, validation collaborative
Mesurer : Indicateurs de satisfaction du besoin, base des critères d'acceptation

---

Phrase repère : *SENSE ne recueille pas des exigences — il construit de la compréhension. La différence entre les deux est la différence entre écrire ce qu'on nous dit et savoir ce qu'on nous dit.*

CHAPITRE 5

Outils et pratiques de SENSE

Atelier d'élicitation ToTAL : guide animateur pas à pas

L'atelier d'élicitation ToTAL est le principal outil opérationnel de la phase SENSE. Il est conçu pour des groupes de cinq à douze participants — un mélange d'utilisateurs finaux, de représentants métier et de membres de l'équipe de développement. Sa durée optimale est de trois à quatre heures, avec deux pauses structurées. Il peut être réalisé en présentiel ou en distanciel, mais exige dans les deux cas un animateur formé à la posture d'élicitation ToTAL.

L'atelier se déroule en cinq séquences. La première séquence — **Ancrage contextuel** (vingt minutes) — consiste à partager avec tous les participants une vue commune du contexte : qui sont les utilisateurs finaux, quel est leur travail réel, quels problèmes ils rencontrent aujourd'hui. Cette séquence utilise souvent une carte d'empathie ou un tableau de contexte préparé avant l'atelier. La deuxième séquence — **Collecte libre** (trente minutes) — invite chaque participant à exprimer, sur des post-its ou dans un outil collaboratif, les besoins tels qu'ils les perçoivent. L'animateur observe et note les formulations, sans intervention.

La troisième séquence — **Clarification des formulations** (quarante-cinq minutes) — est le cœur de l'atelier. L'animateur regroupe les besoins exprimés par thème et invite le groupe à clarifier chaque formulation ambiguë. Les questions d'animateur sont standardisées dans ToTAL : « Qu'est-ce que cela veut dire concrètement pour un utilisateur ? », « Dans quelle situation ce besoin se manifeste-t-il ? », « Comment sauriez-vous que ce besoin est satisfait ? ». Ces trois questions — la question de la concrétisation, la question du contexte, la question de la vérification — sont les trois leviers fondamentaux de l'élicitation ToTAL.

La quatrième séquence — **Priorisation qualifiée** (trente minutes) — applique une grille de priorité simple (Valeur × Urgence × Faisabilité) aux besoins clarifiés. Cette priorisation n'est pas définitive — elle sera affinée dans PLAN — mais elle permet

d'identifier les besoins qui méritent d'entrer dans SPECIFY en priorité. La cinquième séquence — **Validation des compréhensions** (vingt minutes) — demande à chaque participant de reformuler dans ses propres mots les trois besoins prioritaires tels qu'ils ont été compris. Les écarts de reformulation révèlent les zones de compréhension incomplète qui doivent être résolues avant de clore l'atelier.

> **📌 À retenir — Les trois questions de l'animateur SENSE**
>
> 1. **Concrétisation :** « Qu'est-ce que cela veut dire pour un utilisateur dans une situation réelle ? »
> 2. **Contexte :** « Dans quelle situation ce besoin se manifeste-t-il ? Quand est-il le plus aigu ? »
> 3. **Vérification :** « Comment sauriez-vous que ce besoin est satisfait ? Quel signe observeriez-vous ? »
>
> Ces trois questions ne sont pas des formules — ce sont des outils de pensée. L'animateur ToTAL les adapte au contexte, mais ne les abandonne jamais.

La grille de qualification des besoins : 12 critères

Après chaque atelier d'élicitation, les besoins recueillis doivent être qualifiés avant de passer à SPECIFY. La grille de qualification ToTAL propose douze critères organisés en quatre dimensions. Chaque critère est évalué sur une échelle de un à cinq, et un besoin doit atteindre une moyenne de trois ou plus dans chaque dimension pour être considéré comme prêt pour SPECIFY.

Dimension	Critère	Description	Score (1-5)
Pertinence	Alignement stratégique	Le besoin contribue à un objectif stratégique documenté	___
Impact utilisateur réel	Le besoin affecte un nombre significatif	___	

Dimension	Critère	Description	Score (1-5)
	d'utilisateurs dans leur travail réel		
Priorité temporelle	Le besoin est urgent ou conditionne d'autres besoins prioritaires	___	
Précision	Non-ambiguïté	Le besoin peut être lu par deux personnes différentes et compris de la même façon	___
Complétude contextuelle	Le contexte d'usage du besoin est décrit de façon suffisante	___	
Atomicité	Le besoin peut être satisfait de façon indépendante (sans dépendre d'autres besoins non qualifiés)	___	
Testabilité	Vérifiabilité	On peut concevoir un test (manuel ou automatisé) qui vérifie la satisfaction du besoin	___
Mesurabilité	Il existe au moins un indicateur quantitatif permettant de mesurer la satisfaction du besoin	___	
Condition d'échec	On peut décrire ce qui se passe si le besoin n'est PAS satisfait (condition d'échec)	___	
Traçabilité	Source identifiée	On peut identifier qui a exprimé ce besoin et dans quel contexte	___
Lien décisionnel	On peut expliquer pourquoi ce besoin a été retenu (et non	___	

Dimension	Critère	Description	Score (1-5)
	écarté)		
Historique des clarifications	Les clarifications apportées au besoin depuis sa première formulation sont documentées	—	

Les signaux SENSE : comment reconnaître qu'un besoin est prêt à être spécifié

Outre la grille formelle de qualification, l'équipe ToTAL développe avec l'expérience une sensibilité aux signaux informels qui indiquent qu'un besoin est ou n'est pas prêt pour SPECIFY. Ces signaux ne remplacent pas la grille — ils la complètent. Parmi les signaux positifs : toute l'équipe peut expliquer le besoin à un nouveau membre sans hésitation ; plusieurs utilisateurs différents décrivent le besoin de façon convergente ; l'équipe peut dessiner un scénario d'usage complet sans zones d'ombre.

🚦 **Signal — SENSE : signes qu'un besoin est PRÊT ou NON PRÊT**

Besoin PRÊT pour SPECIFY

Besoin NON PRÊT (retour en élicitation)

Tous les membres de l'équipe le décrivent de façon convergente

Deux membres de l'équipe le décrivent différemment

Un test peut être imaginé immédiatement

On ne sait pas comment on saurait que c'est satisfait

La source du besoin est identifiée et joignable

Le besoin vient de « on m'a dit que… »

La condition d'échec est claire

On suppose que le besoin est toujours utile

Le lexique utilisé est dans le glossaire ToTAL

Le besoin contient des termes non définis ou ambigus

Anti-patterns SENSE : trop vite, trop tôt, trop certain

Trois anti-patterns récurrents sabotent régulièrement la phase SENSE dans les équipes qui démarrent avec ToTAL. Le premier est **trop vite** : l'atelier d'élicitation est raccourci parce que l'équipe est convaincue qu'elle connaît déjà les besoins. Cette conviction est presque toujours partiellement fausse — et le « partiellement » est ce qui crée les problèmes majeurs. Le deuxième est **trop tôt** : SENSE est clôturé avant que les utilisateurs réels aient été consultés, parce que les délais pressent ou parce que les commanditaires ont présenté leur vision comme définitive. Le troisième est **trop certain** : l'équipe présente les besoins comme établis et résistance à tout questionnement, parce que remettre en question les besoins est perçu comme une perte de temps ou une remise en cause de l'autorité du commanditaire.

⚠️ Symptômes — Anti-patterns SENSE

- **Trop vite :** L'atelier dure moins de deux heures. Aucun utilisateur final n'était présent. Le backlog est écrit pendant l'atelier.
- **Trop tôt :** SENSE est déclaré « complet » avant que les vrais utilisateurs aient été rencontrés. Les besoins proviennent uniquement de documents écrits.
- **Trop certain :** Toute question sur les besoins est répondue par « c'est dans le document de requirements ». La grille de qualification n'est pas utilisée.

Phrase repère : *SENSE mal fait ne produit pas d'ignorance — il produit quelque chose de bien plus dangereux : une certitude mal fondée.*

CHAPITRE 6

SENSE et l'IA : clarifier avant d'amplifier

L'IA dans la phase SENSE : risques et opportunités

L'intelligence artificielle générative — les LLM (Large Language Models), les outils de génération de spécifications, les assistants de rédaction de user stories — est entrée dans les équipes de développement avec une promesse séduisante : accélérer la production. Et cette promesse est tenue, dans un sens technique étroit. Un LLM peut produire en quelques minutes cinquante user stories à partir d'un brief d'une page. Un outil de génération peut transformer un document de requirements en un backlog structuré en quelques heures. La vitesse est réelle.

Mais dans la phase SENSE, la vitesse est précisément le danger. L'IA amplifie ce qu'elle reçoit. Si elle reçoit des besoins clairs, qualifiés et précis, elle produit des spécifications utiles. Si elle reçoit des besoins flous, elle produit des spécifications en apparence précises — bien rédigées, bien structurées, techniquement impeccables — qui sont fondamentalement à côté de la vérité du besoin. Et parce que ces spécifications sont bien rédigées, elles sont plus difficiles à remettre en question que des spécifications manifestement approximatives. L'IA, dans SENSE, peut transformer la flou en certitude trompeuse — avec une vitesse et une qualité de présentation qui découragent le questionnement.

⚠️ Symptômes — L'IA utilisée trop tôt dans SENSE

- L'équipe génère des user stories avec un LLM avant d'avoir rencontré les utilisateurs réels.
- Les spécifications générées par IA sont acceptées sans révision parce qu'elles « semblent correctes ».
- Le vocabulaire des spécifications ne correspond pas au vocabulaire des utilisateurs observés sur le terrain.
- Les besoins générés par IA sont plus nombreux mais moins pertinents que ceux issus de l'élicitation directe.

Comment l'IA amplifie les besoins flous si SENSE est court-circuité

Le principe fondamental de ToTAL pour l'utilisation de l'IA dans le cycle de développement est simple : **clarifier avant d'amplifier**. L'IA est un amplificateur extraordinaire. Comme tout amplificateur, elle amplifie aussi bien le signal que le bruit. Si le signal est clair — si SENSE a produit des besoins qualifiés, précis et mesurables — l'IA peut l'amplifier de façon spectaculairement utile : générer des variantes de spécifications SI/ALORS, proposer des cas de test, identifier des scénarios d'usage alternatifs. Mais si le signal est flou — si SENSE a été court-circuité — l'IA amplifie le flou, le rend plus volumineux, mieux présenté et plus difficile à corriger.

Dans DREAM-TEAM, après l'épisode ORIGINALIUM raconté au chapitre précédent, l'équipe a été tentée de réaccélérer en utilisant un LLM pour générer un nouveau backlog à partir des observations terrain. Guillaume, le Scrum Master, a posé la question décisive : « Nos observations terrain sont-elles qualifiées selon la grille SENSE ? » La réponse était non. Le groupe a décidé de compléter d'abord la grille de qualification, puis de soumettre les besoins qualifiés au LLM. Le résultat a été frappant : les spécifications générées par l'IA à partir des besoins qualifiés étaient significativement plus pertinentes, plus précises et plus directement utilisables que celles générées à partir des besoins bruts.

> **📌 À retenir — ToTAL et l'IA dans SENSE**
>
> L'IA est un outil de SPECIFY, pas de SENSE. Dans SENSE, l'IA peut aider à organiser, regrouper et reformuler — mais elle ne peut pas remplacer la compréhension humaine du besoin. Utilisez l'IA dans SENSE uniquement pour des tâches d'organisation et de synthèse, jamais pour des tâches de compréhension et de qualification. La compréhension appartient aux humains — et c'est une bonne chose.

Cela dit, l'IA dispose de points d'entrée légitimes dans SENSE. Elle peut analyser des corpus de verbatims d'entretiens pour identifier des patterns récurrents. Elle peut proposer des questions d'approfondissement à partir des premières formulations de besoins. Elle peut générer des personas ou des scénarios d'usage préliminaires qui seront validés lors des ateliers. Ces usages sont légitimes et utiles, à condition que les sorties de l'IA soient traitées comme des hypothèses à valider, pas comme des vérités à accepter.

Phrase repère : *L'IA dans SENSE, c'est comme mettre un amplificateur sur un micro avant d'avoir accordé l'instrument — tout ce qui sortira sera plus fort, mais pas plus juste.*

Partie 3 — SPECIFY : Les spécifications vivantes SI/ALORS

Nommer avant de construire · Douze à quinze pages · Trois chapitres

---

« Une spécification qui ne peut pas être testée n'est pas une spécification — c'est un vœu pieux habillé en exigence. »

CHAPITRE 7

Le langage naturel de la qualité testable

Pourquoi les spécifications meurent et comment les ressusciter

Les spécifications logicielles ont une mauvaise réputation — souvent méritée. Dans de nombreuses organisations, elles sont perçues comme des artefacts bureaucratiques : des documents épais, difficiles à maintenir, que personne ne lit vraiment et qui deviennent rapidement obsolètes dès que le développement commence. Cette perception a conduit une partie du mouvement Agile à abandonner la spécification formelle au profit de la conversation directe et des user stories minimales. Et il y a de la sagesse dans cette posture : une conversation vaut souvent mieux qu'un document de trois cents pages que personne n'a lu.

Mais l'abandon des spécifications formelles a créé un autre problème, plus insidieux : sans spécifications précises et vérifiables, il n'y a pas de base commune pour savoir si ce qui a été développé correspond à ce qui était attendu. La conversation remplace la documentation — mais les conversations ne sont pas vérifiables. Elles ne produisent pas de tests. Elles ne permettent pas de tracer la chaîne de décisions qui a conduit à une implémentation particulière. Et quand un défaut apparaît en production, elles ne permettent pas de savoir si c'est un bug ou un malentendu.

ToTAL propose une voie entre ces deux extrêmes : les **spécifications vivantes**. Une spécification vivante n'est pas un document figé — c'est un artefact dynamique qui évolue avec la compréhension de l'équipe, qui est directement liée aux tests, et qui

reflète en temps réel l'état de la compréhension partagée entre les parties prenantes et l'équipe de développement. La spécification vivante est rédigée dans un langage naturel structuré — le format SI/ALORS — qui la rend à la fois lisible par des non-techniciens et directement exécutable comme test automatisé.

Le format SI/ALORS : définition, exemples, contre-exemples

Le format SI/ALORS est la syntaxe de base de la spécification vivante ToTAL. Sa structure générale est : **SI [condition ou contexte] ALORS [comportement attendu du système]**. Cette structure force simultanément trois qualités fondamentales : la précision (on doit nommer exactement la condition et exactement le comportement attendu), la testabilité (on peut construire un test qui vérifie que la condition produit bien le comportement), et la traçabilité (on peut relier chaque spécification au besoin SENSE dont elle est issue).

Il est important de noter que le format SI/ALORS de ToTAL est inspiré de la syntaxe Gherkin (Given/When/Then) du Behavior-Driven Development, mais il s'en distingue sur un point crucial : ToTAL y intègre systématiquement la dimension de mesure et la condition d'échec, qui sont souvent absentes des scénarios Gherkin classiques.

Type	Exemple	Analyse
Spécification SI/ALORS valide	*SI un enseignant soumet un devoir pour un groupe d'élèves de vingt personnes ou moins, ALORS le système enregistre le devoir et envoie une notification à chaque élève dans un délai inférieur à trente secondes.*	Condition précise (groupe ≤20), comportement précis (enregistrement + notification), mesure incluse (<30s), testable automatiquement.
Spécification vague (à corriger)	*Le système doit être rapide et envoyer des notifications aux élèves.*	« Rapide » est non mesurable. « Des notifications » est non précisé. Condition manquante. Non testable.
Spécification	*SI un enseignant tente*	Condition d'échec explicite,

Type	Exemple	Analyse
SI/ALORS avec condition d'échec	*de soumettre un devoir sans date d'échéance, ALORS le système affiche un message d'erreur explicite et empêche la soumission.*	comportement attendu précis, testable, traçable au besoin de validation des données.
Spécification SI/ALORS avec mesure	*SI un administrateur génère un rapport de participation mensuel pour l'ensemble du réseau d'écoles, ALORS le rapport est disponible en moins de cinq secondes et contient les données de toutes les écoles actives au cours du mois.*	Mesure de performance incluse (<5s), périmètre précis (toutes les écoles actives), testable en performance et en intégrité de données.

Spécifications vivantes : le contrat entre clarté et qualité

Une spécification vivante est plus qu'une spécification bien rédigée. C'est un contrat — au sens social du terme, pas au sens légal. Un contrat entre clarté et qualité : l'équipe s'engage à construire exactement ce qui est spécifié, et les parties prenantes s'engagent à considérer que ce qui est spécifié reflète fidèlement ce qu'elles veulent. Ce contrat n'est valide que si la spécification est suffisamment précise pour être vérifiable, et suffisamment lisible pour que les parties prenantes puissent confirmer qu'elle reflète bien leur intention.

La « vivance » de la spécification tient à deux caractéristiques. Premièrement, elle est maintenue à jour : quand le besoin évolue, la spécification évolue avec lui, et l'historique des changements est documenté. Deuxièmement, elle est directement liée aux tests : chaque spécification SI/ALORS génère au moins un test automatisé qui la vérifie. Quand le test passe, la spécification est satisfaite. Quand le test échoue, la spécification n'est pas satisfaite — et l'équipe sait exactement quoi corriger.

> **📌 À retenir — La spécification vivante ToTAL**
>
> Une spécification vivante ToTAL possède cinq propriétés : elle est **précise** (une seule interprétation possible), **testable** (un test peut la vérifier), **traçable** (liée à un besoin SENSE identifié), **mesurable** (contient au moins un indicateur de mesure) et **vivante** (maintenue à jour avec l'évolution du besoin). Si l'une de ces propriétés manque, la spécification est incomplète.

Phrase repère : *Une spécification qui ne peut pas être testée n'est pas une spécification — c'est une intention habillée en exigence, qui attend patiemment de devenir un problème de production.*

CHAPITRE 8

Écrire des spécifications qui survivent au réel

Atelier SI/ALORS : exercices guidés pour équipes

L'atelier SI/ALORS ToTAL est l'outil opérationnel central de la phase SPECIFY. Il est conçu pour des équipes de développement qui souhaitent apprendre à rédiger des spécifications vivantes à partir de besoins qualifiés issus de SENSE. La durée optimale est de deux heures pour une première session, avec un corpus de dix à quinze besoins qualifiés à spécifier. L'atelier peut être répété en format hebdomadaire de quarante-cinq minutes une fois que l'équipe a intériorisé la pratique.

La session se déroule en trois phases. La première — **Décomposition du besoin** (trente minutes) — consiste à prendre chaque besoin qualifié issu de SENSE et à l'analyser selon trois questions : Quelles sont les conditions dans lesquelles ce besoin se manifeste ? Quel est le comportement exact que le système doit produire dans ces

conditions ? Quel est l'indicateur qui permet de mesurer si ce comportement est satisfaisant ? La deuxième phase — **Rédaction SI/ALORS en binôme** (quarante-cinq minutes) — demande à chaque binôme de rédiger deux à trois spécifications SI/ALORS pour les besoins qui leur ont été attribués. La règle est stricte : chaque spécification doit être rédigée de façon à ce qu'un testeur puisse écrire un test sans poser de questions supplémentaires. La troisième phase — **Revue croisée** (trente minutes) — fait réviser chaque spécification par un binôme différent, selon une grille de révision structurée.

> **🌱 Terrain — DREAM-TEAM : l'atelier SI/ALORS de la semaine 4**
>
> Après avoir refait leur SENSE correctement (semaines 2 et 3), DREAM-TEAM a organisé son premier atelier SI/ALORS formel. Fatima (QA Lead) a animé la session. Le premier besoin à spécifier était : « Les enseignants doivent pouvoir suivre la progression de leurs élèves. » En trente minutes de décomposition, l'équipe a découvert que ce besoin en contenait en réalité sept besoins distincts — suivre la progression en temps réel vs. hebdomadaire, par élève vs. par classe, par compétence vs. par devoir. Ces sept besoins distincts ont généré quatorze spécifications SI/ALORS, dont onze ont passé la revue croisée du premier coup. Les trois qui n'ont pas passé la revue avaient une condition imprécise ou un indicateur manquant. En deux heures, DREAM-TEAM a produit plus de clarté sur le besoin de « suivi de progression » qu'en trois semaines de backlog traditionnel.

Les 10 pathologies de la spécification : symptômes et remèdes

#	Pathologie	Symptôme	Remède ToTAL
1	**Le vœu pieux**	« Le système doit être performant / convivial / fiable »	Ajouter une mesure : « dans un délai inférieur à X », « pour des utilisateurs de niveau Y »

#	Pathologie	Symptôme	Remède ToTAL
2	**La condition fantôme**	La spécification décrit un comportement sans condition (quand ? dans quelles circonstances ?)	Reformuler avec la question : « Dans quelle situation ce comportement doit-il se produire ? »
3	**L'ambiguïté de vocabulaire**	Les termes clés de la spécification ont plusieurs significations possibles	Référencer le Lexique Vivant ToTAL ; définir les termes avant de spécifier
4	**La spécification composite**	Une seule spécification couvre plusieurs comportements distincts (reconnaissable au « et » dans le ALORS)	Diviser en spécifications atomiques : une condition → un comportement → un test
5	**La spécification orpheline**	La spécification ne peut pas être tracée à un besoin SENSE qualifié	Identifier la source ; si aucune source : retourner en SENSE pour valider le besoin
6	**L'implémentation déguisée**	La spécification décrit comment le système doit fonctionner, pas ce qu'il doit produire	Reformuler en termes de comportement observable par l'utilisateur, pas de mécanique interne
7	**La négation sans alternative**	La spécification dit ce que le système ne doit pas faire, sans dire ce qu'il doit faire	Reformuler toujours en termes positifs : SI X ALORS le système fait Y (et non « ne fait pas Z »)
8	**La spécification non maintenue**	La spécification a été modifiée verbalement mais pas mise à jour dans le document	Processus de mise à jour obligatoire : toute modification verbale doit être documentée dans les 24h
9	**La sur-spécification**	Le niveau de détail de la spécification dépasse ce qui est nécessaire pour écrire un test utile	Appliquer le principe de la spécification minimale suffisante : assez précis pour être testable, pas plus

#	Pathologie	Symptôme	Remède ToTAL
10	**La spécification présupposée**	La spécification suppose un comportement du système qui n'a jamais été développé ou validé	Valider les prérequis techniques avant de spécifier ; ajouter les dépendances dans PLAN

La matrice de traçabilité besoin → spécification → test

La matrice de traçabilité est l'outil de gouvernance central de la phase SPECIFY. Elle permet à l'équipe de s'assurer que chaque besoin qualifié issu de SENSE a été traduit en au moins une spécification SI/ALORS, et que chaque spécification est couverte par au moins un test. La matrice de traçabilité est un outil vivant — elle est maintenue à jour tout au long du cycle et constitue l'une des preuves de qualité de la phase SPECIFY.

ID Besoin	Formulation besoin (SENSE)	ID Spec SI/ALORS	ID Test associé	Statut
B-001	Suivi de progression élève par compétence	S-001a, S-001b, S-001c	T-101, T-102, T-103	✅ Couvert
B-002	Notification aux élèves lors d'un devoir	S-002a	T-201	✅ Couvert
B-003	Rapport mensuel de participation réseau	S-003a, S-003b	T-301	⚠ Partiellement couvert
B-004	Accès hors-ligne aux cours pour élèves	Non encore rédigée	—	❌ Non couvert

Phrase repère : *La matrice de traçabilité ne protège pas l'équipe contre les erreurs — elle les rend visibles avant qu'elles ne deviennent des bugs de production.*

CHAPITRE 9

SPECIFY et la dette conceptuelle

Quand les mots flous produisent des spécifications fragiles

La dette technique est un concept bien établi dans le monde du développement logiciel : le coût différé des décisions de conception qui sacrifient la qualité à la vitesse. Mais il existe un autre type de dette, moins souvent nommé et pourtant tout aussi coûteux : la **dette conceptuelle**. La dette conceptuelle se crée quand les concepts fondamentaux du domaine — les mots, les définitions, les distinctions — ne sont pas clarifiés avant d'être utilisés dans les spécifications. Ces mots flous, ces définitions ambiguës, ces distinctions non faites : ils produisent des spécifications fragiles qui s'effondrent dès qu'on les confronte au réel.

Dans la mission ORIGINALIUM, DREAM-TEAM a rencontré un exemple classique de dette conceptuelle. Le besoin B-003 — « rapport mensuel de participation réseau » — contenait trois termes non définis : « mensuel » (mois calendaire ou mois glissant de trente jours ?), « participation » (connexion à la plateforme, ou soumission d'un devoir, ou les deux ?) et « réseau » (toutes les écoles ou seulement les écoles actives au cours du mois ?). Ces trois ambiguïtés ont généré trois implémentations contradictoires dans l'équipe de développement, qui n'ont été découvertes qu'à la revue de code de la semaine 8 — et qui ont nécessité une semaine de correction.

La qualité des concepts comme prérequis de la qualité des spécifications

ToTAL pose un principe simple mais exigeant : **on ne peut pas spécifier ce qu'on n'a pas d'abord défini**. La qualité des spécifications est plafonnée par la qualité des concepts qui les composent. Un concept non défini dans une spécification est une bombe à retardement — il explosera au moment où deux membres de l'équipe découvriront qu'ils n'avaient pas la même définition du mot. Ce moment se produit rarement dans SPECIFY : il se produit presque toujours dans FORGE, lors d'une revue de code, ou dans DELIVERY, lors d'une démo. Plus le moment est tardif, plus la correction est coûteuse.

La solution n'est pas de définir tous les termes de tous les domaines avant de commencer à spécifier — ce serait une tâche infinie et contre-productive. La solution est de maintenir un **Lexique Vivant ToTAL** : un glossaire collaboratif, maintenu par l'équipe, qui définit précisément les termes clés du domaine au fur et à mesure qu'ils apparaissent dans les spécifications. Chaque terme qui entre dans le lexique est accompagné de sa définition, de son contexte d'usage, de ses synonymes acceptés et de ses faux-amis à éviter.

Lexique vivant ToTAL : construire et maintenir

🔭 **Repères — Règles du Lexique Vivant ToTAL**

- Tout terme utilisé dans une spécification SI/ALORS doit être présent dans le lexique ou y être ajouté dans les 24h suivant son utilisation.
- La définition d'un terme doit être rédigée dans le langage des utilisateurs, pas des développeurs.
- Tout changement de définition doit être documenté avec sa date, son auteur et la raison du changement.
- Le lexique est accessible à tous : développeurs, testeurs, Product Owner,

parties prenantes métier.

- Un terme dont la définition est contestée est mis en *status « en discussion »* jusqu'à résolution consensuelle.
- Le lexique est révisé intégralement à chaque début de phase SENSE du cycle suivant.

Le Lexique Vivant ToTAL n'est pas un document à lire — c'est un outil à utiliser activement. Il est la mémoire sémantique de l'équipe. Il capture les décisions de définition qui ont été prises, et protège contre les dérapages conceptuels qui surviennent inévitablement quand les équipes évoluent, quand les membres changent, ou quand les domaines s'élargissent. Dans les organisations qui maintiennent un Lexique Vivant ToTAL sur plusieurs cycles, on observe une réduction mesurable du temps consacré aux clarifications tardives — et une amélioration progressive de la qualité des spécifications de cycle en cycle.

Phrase repère : *La dette conceptuelle est la mère de toutes les dettes logicielles — elle se contracte dans le silence des mots non définis et se rembourse dans la douleur des malentendus de production.*

Partie 4 — PLAN : Décider avec lucidité

Décider avant d'exécuter · Huit à dix pages · Deux chapitres

« Planifier, c'est décider maintenant ce qu'on ne regrettera pas d'avoir décidé plus tard. »

CHAPITRE 10

La planification comme acte de maîtrise

Ce que planifier veut dire dans ToTAL : décider, prioriser, protéger

La planification souffre d'une double réputation difficile. D'un côté, les tenants du management traditionnel la voient comme le fondement du contrôle : un plan détaillé permet de prévoir, de mesurer et de corriger les écarts. De l'autre côté, une partie du mouvement Agile s'est construite en réaction contre la sur-planification : « Répondre au changement plutôt que suivre un plan. » Ces deux positions contiennent de la vérité et de l'excès. ToTAL propose une troisième voie : la planification comme **acte de maîtrise**.

La planification comme acte de maîtrise, c'est décider avec lucidité de ce qu'on va faire, dans quel ordre, avec quels critères de qualité, et avec quelle capacité à ajuster. Ce n'est pas promettre de livrer exactement ce qui est planifié — c'est s'engager à livrer avec la qualité planifiée, et à notifier immédiatement quand un ajustement est nécessaire. La différence est cruciale : dans la planification traditionnelle, l'engagement porte sur le contenu. Dans ToTAL, l'engagement porte sur la qualité du processus de livraison.

ToTAL distingue rigoureusement le **Plan** (le document ou l'artefact qui capture les décisions de planification) de la **planification** (l'activité continue d'adaptation et d'ajustement du plan à la réalité). Un Plan sans planification est un document mort. Une planification sans Plan est une navigation sans carte. Les deux sont nécessaires — et leur articulation est l'un des signes de maturité d'une équipe ToTAL.

Plan vs. planification : la différence qui compte

Dimension	Plan	Planification
Nature	Artefact (document, tableau, canevas)	Activité continue
Fréquence de mise à jour	À chaque cycle ou PI	À chaque itération, événement ou décision significative
Producteur principal	Équipe + Product Owner	Équipe + Scrum Master / Coach ToTAL
Ce qu'il capture	Décisions, priorités, risques, engagements	Ajustements, apprentissages, replanifications
Ce qu'il produit	Clarté sur ce qu'on va construire et comment	Adaptation en temps réel à la réalité
Erreur fatale	Le traiter comme immuable	L'exercer sans référence au Plan

Signaux de planification saine : 8 indicateurs

Une planification saine dans ToTAL produit des signaux observables. Ces signaux ne sont pas des métriques de reporting — ce sont des indicateurs qualtatifs et quantitatifs que l'équipe peut observer dans son propre comportement et dans la qualité de ses livrables.

🔭 Repères — 8 indicateurs de planification saine

1. **Toutes les spécifications retenues dans le plan ont une preuve de qualification SENSE.**
2. **Les risques qualité sont identifiés, évalués et ont un plan de mitigation documenté.**
3. **Les dépendances entre spécifications sont visibles dans le plan.**
4. **La capacité de l'équipe (en jours-personne disponibles) est calculée honnêtement.**

5. **Les critères de sortie de chaque phase sont définis avant de commencer la phase.**

6. **Le RACI de qualité (qui est responsable de quoi en matière de qualité) est documenté.**

7. **Le plan peut être ajusté sans repartir de zéro — il est conçu pour l'adaptation.**

8. **L'équipe peut expliquer oralement le plan à un nouveau membre en moins de vingt minutes.**

Ces huit indicateurs fonctionnent aussi comme grille de diagnostic : si une équipe ne peut pas satisfaire plus de cinq de ces indicateurs, sa planification est insuffisante et le cycle court un risque de qualité significatif. Il ne s'agit pas de punir, mais de nommer : une planification insuffisante est une dette qui se remboursera dans FORGE ou DELIVERY, avec des intérêts.

Phrase repère : *Planifier sans intégrer la qualité, c'est promettre une livraison que l'on ne sait pas encore si l'on sera fier d'avoir faite.*

CHAPITRE 11

Outils de planification ToTAL

Le canevas de planification par phase

Le canevas de planification ToTAL est un outil structuré qui capture, pour chaque phase du cycle, les six décisions fondamentales de la planification : ce qu'on va construire, dans quel ordre, avec quels critères de qualité, avec quelle capacité, avec quels risques identifiés, et avec quels critères de sortie. Il est conçu pour tenir sur une

page — parce qu'un plan qu'on ne peut pas lire en cinq minutes est un plan qu'on ne lira pas.

Section du canevas	Questions clés	Outil recommandé
Périmètre	Quelles spécifications sont incluses dans ce plan ? Lesquelles sont explicitement exclues ?	Matrice de traçabilité SPECIFY
Séquence	Dans quel ordre les spécifications seront-elles implémentées ? Quelles dépendances techniques ?	Graphe de dépendances, backlog ordonné
Qualité intégrée	Quels critères de qualité devront être satisfaits à chaque étape ? Qui vérifie quoi ?	RACI qualité, définition de « done »
Capacité réelle	Combien de jours-personne sont disponibles ? Quelles compétences critiques ? Quels absences prévues ?	Calendrier de capacité
Risques qualité	Quels risques peuvent compromettre la qualité planifiée ? Probabilité × Impact × Mitigation ?	Registre des risques ToTAL
Critères de sortie	Comment saurons-nous que cette phase est complète avec la qualité requise ?	Checklist de sortie de phase

La gestion des risques qualité intégrée au plan

Dans la planification traditionnelle, la gestion des risques est souvent une activité séparée — un registre de risques tenu par un chef de projet, mis à jour en début de projet et consulté rarement. Dans ToTAL, la gestion des risques qualité est intégrée au canevas de planification et est mise à jour à chaque réunion de planification itérative. La distinction importante est entre les risques de délai (qui font partie de la gestion de projet traditionnelle) et les risques de qualité (qui sont la responsabilité spécifique de ToTAL).

Un risque de qualité est une condition qui, si elle se réalise, compromet la qualité de ce qui sera livré — pas seulement le délai de livraison. Par exemple : un développeur clé qui quitte l'équipe est un risque de délai et potentiellement un risque de qualité si ce développeur était le gardien de la connaissance du domaine. Un changement de spécification tardif est un risque de qualité si les tests ne sont pas mis à jour en même temps. Une dépendance technique non validée est un risque de qualité si elle conditionne le comportement d'une fonctionnalité critique.

RACI et responsabilités qualité dans le plan

Le RACI de qualité ToTAL est une version adaptée du modèle RACI (Responsible, Accountable, Consulted, Informed) spécifiquement conçue pour clarifier les responsabilités qualité dans chaque phase du cycle. Contrairement au RACI traditionnel qui porte sur les livrables, le RACI qualité ToTAL porte sur les *preuves de qualité* de chaque phase.

Preuve de qualité	Responsible (R)	Accountable (A)	Consulted (C)	Informed (I)
Grille de qualification des besoins (SENSE)	QA Lead	Product Owner	Équipe, Utilisateurs	Commanditaire
Spécifications SI/ALORS validées (SPECIFY)	QA Lead + Dev Lead	Product Owner	Équipe complète	Commanditaire
Canevas de planification (PLAN)	Scrum Master	Dev Lead	Product Owner, Équipe	Commanditaire
Couverture de tests automatisés (FORGE)	Développeurs	Dev Lead	QA Lead	Product Owner
Critères de sortie satisfaits (DELIVERY)	QA Lead	Product Owner	Dev Lead, Scrum Master	Commanditaire
Rapport d'apprentissages post-livraison	Scrum Master	Product Owner	Équipe complète	Commanditaire

Preuve de qualité	Responsible (R)	Accountable (A)	Consulted (C)	Informed (I)
(CARE)				

❓ Questions d'équipe — PLAN

- Avons-nous un RACI qualité documenté et compris par tous les membres de l'équipe ?
- Nos critères de sortie de phase sont-ils définis avant de commencer la phase — ou pendant ?
- Distinguons-nous clairement nos risques de délai de nos risques de qualité dans notre registre des risques ?
- Notre plan peut-il être expliqué à un nouveau membre de l'équipe en moins de vingt minutes ?

Phrase repère : *Un plan qui n'intègre pas la qualité comme dimension de première classe n'est pas un plan de développement — c'est un plan de production, et la différence entre les deux est la différence entre apprendre et répéter.*

Partie 5 — FORGE : Construire avec qualité intégrée

Construire avant de diffuser · Dix à douze pages · Trois chapitres

---

« La qualité n'est pas ce qu'on vérifie après avoir construit — c'est ce qu'on décide de construire avant de commencer. »

CHAPITRE 12

Transformer la clarté en construction fiable

Qu'est-ce que FORGE dans le SDLC ToTAL ?

FORGE est la phase du cycle où la clarté produite par SENSE et SPECIFY, et la lucidité produite par PLAN, se transforment en logiciel fonctionnel. Le nom FORGE est choisi avec intention : une forge est un lieu de travail intense, physique, exigeant — mais c'est aussi un lieu de précision. Le forgeron ne frappe pas au hasard ; chaque coup est délibéré, calculé, ajusté. FORGE dans ToTAL, c'est construire avec la même intentionnalité : chaque ligne de code, chaque revue, chaque test est un geste délibéré au service d'une qualité planifiée.

La différence fondamentale entre FORGE dans ToTAL et le « développement » dans les approches classiques réside dans la relation entre la construction et la vérification. Dans les approches classiques, la vérification vient après la construction : on développe, puis on teste, puis on corrige. Dans ToTAL, la vérification est contemporaine de la construction : les tests sont écrits avant le code (TDD), les comportements sont spécifiés avant d'être implémentés (BDD), et la qualité est vérifiée à chaque intégration (CI). La qualité est bâtie, pas contrôlée après coup — et cette distinction change fondamentalement la dynamique du développement.

La qualité bâtie, pas contrôlée après coup

La distinction entre qualité bâtie et qualité contrôlée est l'une des plus importantes de ToTAL. Elle reflète une philosophie profonde sur la nature du développement logiciel : la qualité n'est pas une propriété qu'on ajoute à un produit fini — c'est une propriété qui doit être présente à chaque étape de sa construction. Un logiciel qui est bien conçu, bien testé et bien intégré tout au long de son développement ne nécessite pas

une phase de QA intensive en fin de cycle pour découvrir des problèmes. Les problèmes sont détectés et corrigés au moment où ils sont introduits — lorsque le coût de la correction est minimal.

Cette philosophie est soutenue par des décennies de recherche en ingénierie logicielle. La règle empirique du « coût de correction exponentiel » — souvent attribuée à Barry Boehm, qui l'a formalisée dans les années 1970 et dont les travaux ultérieurs ont affiné les chiffres — indique que le coût de correction d'un défaut croît à mesure qu'il est découvert tard dans le cycle. Un défaut découvert pendant la conception coûte environ un pour corriger. Ce même défaut, découvert pendant les tests systèmes, coûte entre dix et cent fois plus cher. Découvert en production, il peut coûter plusieurs centaines de fois plus cher, en incluant les coûts de remédiation, d'impact utilisateur et de réputation. La qualité bâtie n'est pas une aspiration morale — c'est une stratégie économique.

Automatisation et qualité : le rôle de BEASTEST dans FORGE

BEASTEST — l'acronyme ToTAL pour le cadre d'automatisation des tests (Build, Execute, Assess, Store, Track, Evolve, Systematize, Transfer) — est le système d'automatisation qui soutient la qualité bâtie dans FORGE. BEASTEST n'est pas un outil spécifique — c'est un ensemble de principes et de pratiques qui gouvernent comment l'automatisation des tests est construite, maintenue et utilisée dans le SDLC ToTAL.

Le principe fondamental de BEASTEST est que les tests automatisés ne sont pas des outils de vérification — ce sont des outils de documentation exécutable. Chaque test automatisé dans ToTAL est la version exécutable d'une spécification SI/ALORS. Il ne vérifie pas seulement que le code fait quelque chose — il vérifie que le code fait exactement ce qui a été spécifié dans SPECIFY. Cette correspondance directe entre spécifications et tests est ce qui fait de la suite de tests ToTAL une documentation vivante du comportement attendu du système.

> **📌 À retenir — FORGE et la qualité bâtie**
>
> Dans ToTAL, un test automatisé est la version exécutable d'une spécification SI/ALORS. Si la spécification change, le test change. Si le test échoue, la spécification n'est pas satisfaite. Cette correspondance directe transforme la suite de tests en une documentation vivante du comportement du système — toujours à jour, toujours vérifiable, toujours honnête.

Phrase repère : *FORGE sans qualité intégrée, c'est construire vite une maison sans fondations — impressionnant à regarder, dangereux à habiter.*

CHAPITRE 13

Pratiques de FORGE

TDD, BDD et spécifications exécutables dans FORGE

Le Test-Driven Development (TDD) et le Behavior-Driven Development (BDD) ne sont pas des inventions de ToTAL — ce sont des pratiques éprouvées de l'industrie du développement logiciel, popularisées respectivement par Kent Beck (TDD, 2002) et Dan North (BDD, 2006). ToTAL les intègre et les articule dans le cadre du cycle, en les connectant explicitement à la phase SPECIFY et au format SI/ALORS.

Dans ToTAL, le TDD est la pratique par laquelle le développeur écrit d'abord un test qui vérifie le comportement attendu de l'unité de code qu'il va écrire, puis écrit le code minimal pour faire passer ce test, puis refactorise le code pour améliorer sa qualité interne. Le cycle TDD — Rouge (test échoue), Vert (test passe), Refactorise — est une micro-boucle d'apprentissage intégrée dans chaque session de

développement. Chaque itération de ce cycle produit à la fois du code fonctionnel et un test qui documente ce comportement.

Le BDD étend le TDD en montant d'un niveau d'abstraction : les comportements spécifiés dans le format Gherkin (Given/When/Then) — qui est très proche du format SI/ALORS de ToTAL — sont directement transformables en tests automatisés par des outils comme Cucumber, SpecFlow ou Behave. Dans ToTAL, les spécifications SI/ALORS rédigées dans SPECIFY sont utilisées comme base directe pour les scénarios BDD de FORGE. Cette correspondance directe élimine le risque de désynchronisation entre spécifications et tests — un risque majeur dans les approches où la spécification et le test sont écrits par des personnes différentes, à des moments différents, sans référence commune.

Les revues de code comme acte qualité : guide animateur

La revue de code est l'une des pratiques les plus universellement présentes dans les équipes de développement — et l'une des moins bien pratiquées. Dans beaucoup d'équipes, la revue de code est une formalité : quelqu'un approuve un pull request en cinq minutes sans l'avoir vraiment lu, pour satisfaire une règle de processus. Dans ToTAL, la revue de code est un acte qualité délibéré : un moment structuré où au moins un pair examine le code non seulement pour identifier des bugs, mais pour vérifier que ce qui a été construit correspond à ce qui a été spécifié.

La revue de code ToTAL suit quatre axes d'évaluation. Le premier axe — **Conformité aux spécifications** — vérifie que le code implémente exactement ce qui a été spécifié dans les SI/ALORS, pas plus, pas moins. Le deuxième axe — **Couverture de tests** — vérifie que chaque comportement implémenté est couvert par au moins un test automatisé. Le troisième axe — **Lisibilité et maintenabilité** — vérifie que le code peut être compris par n'importe quel membre de l'équipe sans explication du développeur auteur. Le quatrième axe — **Gestion des cas limites** — vérifie que les conditions d'échec des spécifications SI/ALORS sont correctement traitées dans le code.

🌱 Terrain — DREAM-TEAM : la revue de code qui a sauvé la semaine 9

En semaine 9 de la mission ORIGINALIUM, David a soumis une pull request pour l'implémentation du rapport mensuel de participation (B-003). La revue de code ToTAL, menée par Élise, a pris quarante-cinq minutes. Elle a identifié quatre problèmes : le code calculait la participation sur le mois calendaire, alors que la spécification S-003a définissait explicitement « mois glissant de trente jours » (grâce au Lexique Vivant ToTAL). Il n'y avait pas de test pour le cas d'un réseau avec aucune école active. La performance n'avait pas été mesurée (critère des cinq secondes de S-003b). Et enfin, un terme dans le commentaire du code utilisait « école » et « établissement » de façon interchangeable, alors que le lexique les distinguait. Ces quatre problèmes, corrigés en deux heures, ont évité ce qui aurait été une démo échouée et probablement une semaine de correction post-livraison.

Intégration continue et signaux de qualité en temps réel

L'intégration continue (CI) est, dans ToTAL, un système de signaux de qualité en temps réel. Chaque fois qu'un développeur intègre son code dans la branche principale, le pipeline CI déclenche automatiquement l'ensemble de la suite de tests, les analyses statiques de code, les vérifications de couverture et les contrôles de conformité. Si l'un de ces signaux passe au rouge, l'équipe est immédiatement alertée — et la règle ToTAL est claire : un pipeline rouge est la priorité numéro un de l'équipe, devant toute nouvelle fonctionnalité.

Cette règle peut sembler contre-intuitive dans un environnement sous pression de délais. Mais elle repose sur une logique solide : un pipeline rouge signifie que quelque chose a été cassé dans le système. Plus longtemps ce quelque chose reste cassé, plus il est susceptible d'être dépendé par d'autres composants qui seront aussi cassés. La dette technique s'accumule exponentiellement si on continue de construire sur une base instable. Corriger immédiatement coûte quelques heures. Corriger après cinq sprints peut coûter plusieurs semaines.

🚦 **Signal — Pipeline CI ToTAL : code couleur**

Vert : Tous les tests passent. Couverture ≥ seuil. Analyses statiques propres. Prêt pour DELIVERY.
Jaune : Tous les tests passent, mais couverture en dessous du seuil ou avertissements statiques. Décision d'équipe requise avant DELIVERY.
Rouge : Au moins un test échoue. Stop immédiat. Correction prioritaire avant tout autre développement.

Phrase repère : *Un test qui échoue n'est pas un problème — c'est une information. Un test qui n'existe pas, c'est une ignorance consentie.*

CHAPITRE 14

Anti-patterns de FORGE

La course : construire vite sans construire bien

L'anti-pattern de la course est le plus courant dans les équipes sous pression de délais. Il se manifeste par une série de compromis qualité successifs, chacun présenté comme « temporaire » ou « acceptable dans le contexte » : les tests sont écrits après le code plutôt qu'avant, ou pas du tout. Les revues de code sont raccourcies ou supprimées pour gagner du temps. Les comportements aux limites ne sont pas testés parce que « ça ne se produira probablement jamais ». Les warnings d'analyse statique sont ignorés parce qu'ils sont « cosmétiques ».

Pris individuellement, chacun de ces compromis semble gérable. Collectivement et cumulés sur plusieurs cycles, ils produisent un système fragile, difficile à maintenir, et qui surprend désagréablement à chaque nouvelle fonctionnalité. L'ironie de la course est qu'elle finit presque toujours par être plus lente que le développement qualité : le

temps gagné sur les compromis est perdu, avec des intérêts, en debugging, en correctifs et en retravailler.

⚠️ **Symptômes — L'anti-pattern de la course dans FORGE**

- La couverture de tests diminue de sprint en sprint.
- Les revues de code durent en moyenne moins de quinze minutes par pull request.
- Le nombre d'incidents de production augmente après chaque livraison.
- L'équipe passe plus de trente pour cent du sprint à corriger des défauts des sprints précédents.
- Les développeurs ne peuvent pas expliquer le comportement de certaines parties du code sans l'examiner directement.

La dette technique comme choix, pas comme accident

ToTAL refuse la vision romantique de la dette technique comme phénomène inévitable et mystérieux qui s'accumule « naturellement » dans tout logiciel vivant. La dette technique est un choix — parfois un choix conscient et justifié, souvent un choix inconscient et regrettable. La distinction importante est entre la dette technique **délibérée** (on sait qu'on accumule de la dette et on a documenté pourquoi, avec un plan de remboursement) et la dette technique **accidentelle** (on accumule de la dette sans le savoir ou sans l'admettre).

ToTAL accepte la dette technique délibérée comme un outil de gestion de projet valide dans certains contextes. Il est parfois raisonnable de sacrifier temporairement la qualité interne d'un composant pour livrer une fonctionnalité critique dans un délai critique. Mais cette décision doit être : documentée (dans le registre des risques qualité du Plan), motivée (avec une justification explicite), limitée dans le temps (avec une date de remboursement ou un critère de déclenchement du remboursement), et

comptabilisée (avec une estimation de l'effort de remboursement incluse dans les plans futurs).

Quand FORGE oublie SENSE et SPECIFY

Le troisième anti-pattern de FORGE est l'oubli de ses origines. Les développeurs, plongés dans les détails techniques de l'implémentation, perdent parfois de vue pourquoi une fonctionnalité existe et comment elle devrait se comporter du point de vue de l'utilisateur. Ce glissement — de l'intention (SENSE et SPECIFY) à la mécanique (FORGE) — est naturel dans le travail d'implémentation intensive. Mais il est aussi dangereux : il conduit à des décisions d'implémentation qui optimisent la technique au détriment de l'expérience utilisateur.

La pratique ToTAL pour prévenir cet oubli est simple : les spécifications SI/ALORS restent affichées (physiquement ou numériquement) dans l'espace de travail de l'équipe tout au long de FORGE. Chaque revue de code commence par la lecture à voix haute de la spécification que le code est censé implémenter. Chaque test automatisé porte le nom de la spécification qu'il vérifie. Ces pratiques peuvent sembler rituelles — mais leur effet sur le maintien du lien entre intention et implémentation est documenté et significatif.

❓ Questions d'équipe — Anti-patterns FORGE

- Pouvons-nous tracer chaque ligne de code à une spécification SI/ALORS ?
- Notre dette technique est-elle délibérée et documentée, ou accidentelle et ignorée ?
- La pression de délai nous conduit-elle à sacrifier des tests ou des revues ? Si oui, le faisons-nous consciemment ?
- Nos développeurs peuvent-ils expliquer le *pourquoi* de chaque fonctionnalité qu'ils développent, pas seulement le *comment* ?

Phrase repère : *FORGE qui oublie SENSE construit avec précision ce qui n'avait pas besoin d'être construit — l'excellence technique au service d'une mauvaise compréhension est un échec que personne ne réclame mais que tout le monde produit.*

Partie 6 — DELIVERY et CARE : Livrer et apprendre

Livrer avec confiance · apprendre après la livraison · Dix à douze pages · Trois chapitres

---

« Livrer n'est pas la fin du cycle — c'est le moment où le cycle devient honnête avec lui-même. »

CHAPITRE 15

DELIVERY : la livraison comme acte de confiance

Qu'est-ce qu'une livraison ToTAL réussie ?

Dans le vocabulaire courant du développement logiciel, une livraison réussie est souvent définie de façon purement technique : le code est déployé en production, le déploiement ne génère pas d'erreur critique, et les systèmes sont fonctionnels. Cette définition est nécessaire mais insuffisante. Dans ToTAL, une livraison réussie satisfait trois critères simultanément. Premièrement, ce qui a été livré correspond à ce qui avait été spécifié — et les preuves de cette conformité sont disponibles (les tests automatisés passants, les revues complétées, les critères de sortie satisfaits).

Deuxièmement, les parties prenantes qui avaient défini les besoins dans SENSE reconnaissent dans ce qui est livré une réponse à leurs vrais besoins. Troisièmement, l'équipe peut défendre la qualité de ce qu'elle livre avec des données — pas des opinions, pas des impressions, mais des données.

Cette définition en trois critères reflète la chaîne Clarté → Qualité → Confiance dans son état le plus tangible. La conformité aux spécifications démontre que la qualité technique est présente. La reconnaissance par les parties prenantes démontre que la clarté initiale était juste. Et la capacité à défendre la qualité avec des données crée la confiance — la confiance que ce qui est livré peut être utilisé, étendu et amélioré sans crainte.

Les critères de sortie par phase : définition et usage

Les critères de sortie (exit criteria) sont l'une des innovations les plus importantes du SDLC ToTAL par rapport aux approches agiles classiques. Dans Scrum, par exemple, la « définition de done » est un critère de sortie pour les user stories — mais elle ne s'applique généralement pas aux phases du cycle lui-même. Dans ToTAL, chaque phase possède ses propres critères de sortie qui doivent être satisfaits avant que la phase ne soit considérée comme terminée et avant que la suivante ne commence.

Phase	Critères de sortie obligatoires	Qui valide
SENSE	100% des besoins prioritaires passent la grille de qualification (score moyen ≥ 3/5 par dimension). Lexique Vivant mis à jour. Sources documentées.	QA Lead + Product Owner
SPECIFY	100% des besoins qualifiés ont au moins une spécification SI/ALORS. Matrice de traçabilité complète. Toutes les specs validées par le Product Owner.	QA Lead
PLAN	Canevas de planification	Dev Lead + Scrum Master

Phase	Critères de sortie obligatoires	Qui valide
	complété. Risques qualité identifiés et mitigés. RACI qualité validé. Critères de sortie FORGE définis.	
FORGE	Couverture de tests ≥ seuil défini. Pipeline CI vert. Revues de code complétées pour 100% des pull requests. Zéro défaut bloquant ou critique ouvert.	QA Lead + Dev Lead
DELIVERY	Déploiement réussi. Tests de fumée (smoke tests) passants en production. Parties prenantes notifiées. Plan de rollback validé.	Dev Lead + Product Owner
CARE	Métriques post-livraison collectées sur au moins deux semaines. Rapport d'apprentissages produit. Intégration dans le prochain SENSE planifiée.	Scrum Master + Product Owner

Le contrat qualité de livraison : qui décide, comment, sur quelle base

Le contrat qualité de livraison ToTAL est un artefact formel — une page maximale — qui documente, pour chaque livraison, trois informations critiques : ce qui est livré (périmètre précis, référençant les spécifications SI/ALORS), les preuves de qualité disponibles (résultats de tests, métriques de couverture, résultats des revues), et la décision de livraison (qui a décidé de livrer, sur la base de quelles preuves, avec quelles réserves éventuelles).

La troisième information — la décision de livraison — est souvent la plus négligée. Dans beaucoup d'organisations, la livraison « arrive » plutôt qu'elle n'est décidée : le pipeline CI/CD pousse automatiquement vers la production quand tous les tests passent. ToTAL ne s'oppose pas à l'automatisation de la livraison — mais il insiste sur le fait que la décision de livrer reste une décision humaine, informée par des données. L'automatisation exécute la décision. Elle ne la remplace pas.

🌱 Terrain — DREAM-TEAM : la livraison de la release 1.0 d'ORIGINALIUM

La première livraison d'ORIGINALIUM — un périmètre de douze fonctionnalités couvrant la gestion des devoirs et le suivi de progression — a failli être livrée sans contrat qualité de livraison. Guillaume a insisté pour que l'équipe complète l'artefact avant le déploiement. Ce processus a pris deux heures — et pendant ces deux heures, l'équipe a découvert que deux des critères de sortie FORGE n'étaient pas satisfaits : la couverture de tests pour le module de notification était à soixante-deux pour cent (sous le seuil de quatre-vingts pour cent), et une revue de code était en attente depuis trois jours. La livraison a été repoussée de quarante-huit heures. Ces quarante-huit heures ont évité ce qui aurait été, selon Fatima, « au moins une semaine de gestion de crise post-livraison ». La confiance du commanditaire dans la première release d'ORIGINALIUM a été totale — une première dans l'histoire récente de l'organisation.

Phrase repère : *Livrer sans preuves, c'est promettre sans garantie — et dans le développement logiciel, les garanties se construisent dans le cycle, pas dans les excuses post-livraison.*

CHAPITRE 16

CARE : transformer la livraison en boucle d'apprentissage

Pourquoi CARE est la capacité la plus négligée

Si SENSE est la plus critique des six capacités du SDLC ToTAL — parce que les erreurs qui y sont commises se propagent dans tout le cycle — CARE est sans conteste la plus

négligée. La raison est simple et humaine : après la livraison, l'équipe est épuisée et pressée de passer au cycle suivant. Les pressions organisationnelles poussent immédiatement vers le prochain sprint, le prochain projet, la prochaine release. CARE est perçu comme un luxe — quelque chose qu'on ferait « si on avait le temps ». Et on n'a jamais le temps.

Cette négligence de CARE est l'une des causes les plus profondes de la stagnation des équipes de développement. Sans CARE, l'équipe ne sait pas si ce qu'elle a livré a produit la valeur espérée. Elle ne sait pas quels aspects de son cycle ont bien fonctionné et méritent d'être renforcés. Elle ne sait pas quels apprentissages post-livraison devraient modifier ses pratiques SENSE du prochain cycle. Elle recommence donc, avec les mêmes forces et les mêmes faiblesses — parfois en sachant qu'elle répète les mêmes erreurs, mais sans les outils pour les corriger systématiquement.

ToTAL traite CARE non pas comme une phase optionnelle, mais comme une condition de santé du cycle. Un cycle sans CARE n'est pas un SDLC ToTAL complet — c'est un SDLS (Software Development Life Sequence) : une séquence d'étapes qui se terminent à la livraison, sans la boucle d'apprentissage qui en fait un vrai cycle. CARE est ce qui transforme une séquence en cycle, une exécution en apprentissage, une livraison en progrès.

Le cycle DIFFUSER-VEILLER-INTERVENIR-RECUEILLIR

CARE dans ToTAL est structuré autour d'un cycle interne de quatre activités : **Diffuser, Veiller, Intervenir, Recueillir**. Ce cycle DVIR est la mécanique opérationnelle de CARE — la façon dont l'équipe transforme la mise en production d'un logiciel en une source active d'apprentissage.

Diffuser : après la livraison, l'équipe s'assure que le produit est effectivement utilisé — par les bons utilisateurs, dans les bonnes conditions, avec le support nécessaire pour une adoption réussie. La diffusion n'est pas une activité passive de déploiement : c'est une activité active de mise en usage. Elle comprend la communication aux utilisateurs, la formation si nécessaire, la documentation de démarrage et le monitoring initial.

Veiller : pendant les deux à quatre premières semaines d'usage, l'équipe surveille activement les métriques post-livraison définies dans PLAN et CARE. Ces métriques couvrent plusieurs dimensions : l'adoption (combien d'utilisateurs utilisent la fonctionnalité ?), la satisfaction (comment les utilisateurs évaluent-ils la fonctionnalité ?), la performance (les temps de réponse sont-ils dans les limites spécifiées ?), et la fiabilité (combien d'incidents la fonctionnalité a-t-elle générés ?).

Intervenir : quand les métriques révèlent un problème — ou une opportunité — l'équipe intervient de façon ciblée et rapide. L'intervention peut être technique (correctif d'un bug identifié par les métriques), fonctionnelle (ajustement d'un comportement qui ne correspond pas à l'usage réel), ou communicationnelle (clarification pour les utilisateurs d'une fonctionnalité mal comprise).

Recueillir : après la période de veille, l'équipe recueille et documente tous les apprentissages produits par l'usage réel. Ces apprentissages sont organisés en quatre catégories : apprentissages sur le besoin (le besoin était-il ce que nous pensions ?), apprentissages sur la spécification (nos spécifications SI/ALORS étaient-elles justes ?), apprentissages sur la construction (notre code reflétait-il fidèlement les spécifications ?), et apprentissages sur le cycle (notre processus SDLC ToTAL a-t-il bien fonctionné dans ce contexte ?).

Métriques post-livraison : signaux utiles vs. métriques de reporting

ToTAL distingue rigoureusement deux types de métriques post-livraison. Les **métriques de signaux utiles** sont celles que l'équipe utilise elle-même pour comprendre et améliorer le produit et le cycle. Les **métriques de reporting** sont celles qui sont produites pour satisfaire une demande de reporting externe — management, comité de pilotage, commanditaire. Les deux types existent dans toute organisation, mais leur valeur est très différente.

Dimension	Métrique de signal utile	Métrique de reporting (à éviter)
Adoption	% d'utilisateurs cibles ayant	Nombre total de

Dimension	Métrique de signal utile	Métrique de reporting (à éviter)
	utilisé la fonctionnalité au moins une fois dans les 7 jours suivant la livraison	connexions à la plateforme
Satisfaction	Score NPS spécifique à la fonctionnalité livrée, avec verbatims	NPS global de la plateforme
Performance	Temps de réponse du rapport mensuel P95 (95e percentile)	Temps de réponse moyen de toutes les API
Fiabilité	Nombre d'incidents liés à la fonctionnalité livrée, par catégorie	Taux de disponibilité global (SLA %)
Usage réel	Parcours utilisateurs les plus fréquents vs. parcours anticipés en SENSE	Pages vues ou événements génériques

---

Phrase repère : *CARE sans métriques de signaux, c'est de l'introspection sans miroir — on se croit amélioré sans pouvoir le vérifier, et on se croit excellent sans pouvoir le démontrer.*

CHAPITRE 17

Boucler le cycle : de CARE à SENSE

Comment les apprentissages post-livraison alimentent le prochain SENSE

La boucle du SDLC ToTAL se ferme au moment où les apprentissages documentés dans CARE alimentent le SENSE du cycle suivant. Cette connexion entre CARE et SENSE est le mécanisme par lequel le cycle devient un vrai système apprenant — un système qui ne se contente pas de se répéter, mais qui s'améliore de cycle en cycle, progressivement et de façon mesurable.

Dans la pratique, cette connexion se réalise à travers un atelier structuré que ToTAL appelle la **Rétrospective de Cycle Enrichie** — à distinguer de la rétrospective de sprint classique. Là où la rétrospective de sprint porte sur le processus de l'équipe au cours du sprint, la Rétrospective de Cycle Enrichie porte sur trois dimensions complémentaires : les apprentissages sur le produit (ce que l'usage réel a révélé sur le besoin et sur la qualité de ce qui a été livré), les apprentissages sur le cycle (ce que ce cycle a révélé sur la maturité des six capacités SDLC ToTAL de l'équipe), et les décisions pour le SENSE suivant (comment les apprentissages de CARE vont modifier l'approche SENSE du prochain cycle).

La Rétrospective de Cycle Enrichie produit un artefact spécifique : le **Mémo d'Alimentation SENSE**. Ce mémo, d'une à deux pages, capture les décisions prises quant à ce qui doit changer dans le prochain SENSE, pourquoi, et comment. Il est remis à l'animateur du prochain atelier d'élicitation ToTAL comme point de départ de la séquence de SENSE. Cette pratique garantit que les apprentissages de CARE ne disparaissent pas dans des notes de rétrospective oubliées — ils sont activement intégrés dans le cycle suivant.

La mémoire organisationnelle ToTAL

À mesure que les cycles s'accumulent, le SDLC ToTAL construit quelque chose que la plupart des organisations ne possèdent pas vraiment : une **mémoire organisationnelle**. La mémoire organisationnelle ToTAL est l'ensemble des apprentissages documentés, qualifiés et indexés, produits par tous les cycles passés d'une équipe — et accessibles pour informer les cycles futurs. Elle comprend le Lexique Vivant ToTAL (qui capture l'évolution des concepts et des définitions), les Mémos d'Alimentation SENSE (qui capturent les décisions de changement de cycle en cycle), les registres des risques qualité (qui capturent les risques identifiés et leur résolution), et les rapports de métriques post-livraison (qui capturent l'impact réel des livraisons).

Une organisation qui maintient cette mémoire dispose d'un avantage compétitif concret : elle peut intégrer un nouveau membre d'équipe sans perte de contexte sur les décisions passées. Elle peut démarrer un nouveau cycle avec une compréhension plus riche du domaine que ce qu'un SENSE « vierge » pourrait produire. Elle peut

identifier des patterns récurrents dans ses difficultés et ses succès — et les adresser de façon systématique plutôt qu'anecdotique.

L'excellence durable comme résultat naturel d'un cycle sain

L'excellence durable — la capacité d'une équipe à livrer régulièrement de la valeur réelle, avec une qualité consistante, sur une longue durée — est le résultat naturel d'un SDLC ToTAL bien pratiqué. Ce n'est pas un objectif à atteindre en soi : c'est un effet émergent d'un cycle qui apprend. Une équipe qui pratique rigoureusement SENSE produit progressivement de meilleures compréhensions du besoin. Une équipe qui pratique rigoureusement SPECIFY produit progressivement des spécifications plus précises et mieux tracées. Une équipe qui pratique rigoureusement FORGE produit progressivement un code plus fiable et plus maintenable. Et une équipe qui pratique rigoureusement CARE devient progressivement plus précise dans sa compréhension de ce que ses utilisateurs ont vraiment besoin.

> **📌 À retenir — L'excellence durable dans ToTAL**
>
> L'excellence durable n'est pas un état — c'est une trajectoire. Une équipe ToTAL n'est pas excellente parce qu'elle ne fait pas d'erreurs : elle est excellente parce qu'elle apprend plus vite de ses erreurs que la plupart des équipes, et qu'elle construit des mécanismes pour que les mêmes erreurs ne se reproduisent pas de cycle en cycle. La différence entre une équipe ordinaire et une équipe excellente n'est pas l'absence de problèmes — c'est la vitesse et la profondeur de l'apprentissage à partir des problèmes.

Phrase repère : *De CARE à SENSE, le cycle boucle — mais il ne revient pas au même point. Il revient à un point légèrement plus haut, légèrement plus sage, légèrement plus capable. C'est cela, la spirale de la maîtrise.*

Partie 7 — Le SDLC ToTAL dans votre contexte

Adapter sans trahir · Six à huit pages · Deux chapitres

« ToTAL n'est pas un cadre rival — c'est une discipline de maîtrise que tout cadre peut accueillir. »

CHAPITRE 18

ToTAL et les cadres existants : Agile, SAFe, DevOps

Pourquoi ToTAL n'est pas un cadre concurrent mais une discipline de maîtrise

Cette question revient souvent lors des premières rencontres avec ToTAL : « Est-ce que ToTAL remplace Scrum ? Est-ce que je dois abandonner SAFe pour adopter ToTAL ? » La réponse est non — et cette réponse est délibérée, pas diplomatique. ToTAL n'est pas conçu pour remplacer les cadres existants. Il est conçu pour les enrichir. La distinction entre un cadre et une discipline de maîtrise est essentielle : un cadre définit des rôles, des cérémonies et des artefacts. Une discipline de maîtrise définit une posture, des principes et des preuves de qualité. Les deux sont compatibles, et souvent complémentaires.

Scrum définit comment une équipe s'organise pour travailler en itérations courtes. ToTAL définit comment une équipe s'assure que ce qu'elle produit dans ces itérations est de la vraie qualité — comprise, spécifiée, construite et livrée avec les preuves qui

le démontrent. On peut pratiquer Scrum parfaitement selon le Scrum Guide et avoir un SDLC ToTAL robuste dans le même temps. Les cérémonies Scrum — Planning, Daily, Review, Retrospective — restent intactes. C'est la qualité de ce qui se passe dans et entre ces cérémonies qui change avec ToTAL.

Intégrer le SDLC ToTAL dans Scrum : guide pratique

Cérémonie Scrum	Apport ToTAL	Changement pratique
Sprint Planning	Vérification que les user stories sélectionnées ont des spécifications SI/ALORS complètes	Ajout d'une vérification de la matrice de traçabilité avant de confirmer le périmètre du sprint
Daily Scrum	Intégration d'un signal de qualité quotidien (pipeline CI vert/rouge)	La 4e question quotidienne : « Y a-t-il un signal qualité (pipeline, test en échec) qui mérite l'attention de l'équipe ? »
Sprint Review	Démonstration basée sur les spécifications SI/ALORS, pas seulement sur les features	Chaque fonctionnalité démontrée est accompagnée de la spécification qu'elle satisfait et du test qui le prouve
Retrospective	Inclusion des indicateurs de santé du cycle dans la rétrospective	Revue des six indicateurs de santé SDLC ToTAL (un par phase) en plus des éléments classiques de la rétrospective
Backlog Refinement	Application de la grille de qualification SENSE et rédaction des specs SI/ALORS	Le refinement ne porte que sur des besoins qui ont passé la grille de qualification SENSE

Intégrer le SDLC ToTAL dans SAFe : adaptations et ponts

SAFe (Scaled Agile Framework) opère à une échelle différente de Scrum — programme, portfolio, large solution — et ToTAL s'y intègre de façon distincte. Dans SAFe, les niveaux d'abstraction sont multiples : les Epics au niveau portfolio, les

Features au niveau programme, les Stories au niveau équipe. Le SDLC ToTAL intervient principalement au niveau équipe, mais il apporte des connexions importantes aux niveaux supérieurs.

Au niveau programme, SENSE correspond à l'analyse des Epics et des Features dans le cadre du PI Planning (Program Increment Planning). L'atelier d'élicitation ToTAL peut être adapté pour fonctionner comme activité préalable au PI Planning, garantissant que les Features présentées lors du PI ont passé une grille de qualification SENSE avant d'être estimées et planifiées. SPECIFY, au niveau programme, se traduit par des Acceptance Criteria au format SI/ALORS pour chaque Feature — une pratique qui améliore significativement la qualité de la communication entre l'ART (Agile Release Train) et les équipes de développement.

📌 À retenir — ToTAL dans SAFe

Dans SAFe, ToTAL opère comme une discipline de qualité transverse à tous les niveaux. SENSE améliore la qualité du PI Planning. SPECIFY améliore la qualité des Features et Acceptance Criteria. PLAN enrichit l'Iteration Planning. FORGE renforce les pratiques Built-In Quality de SAFe. DELIVERY s'aligne avec les System Demos. CARE enrichit les Inspect & Adapt workshops. ToTAL ne remplace pas SAFe — il en améliore la qualité profonde.

Phrase repère : *ToTAL dans Scrum, c'est une âme qui anime le corps. ToTAL dans SAFe, c'est une discipline qui donne de la profondeur à la structure. Le corps et la structure sont nécessaires — mais sans âme et sans profondeur, ils restent vides.*

CHAPITRE 19

Démarrer avec le SDLC ToTAL : premières semaines

La recette de démarrage ToTAL SDLC en 4 étapes

Démarrer avec le SDLC ToTAL ne nécessite pas une transformation organisationnelle de grande envergure. ToTAL est conçu pour être adopté progressivement, phase par phase, en commençant par la capacité qui apportera le plus de valeur dans votre contexte spécifique. La recette de démarrage en quatre étapes est une progression pragmatique qui permet à une équipe d'adopter ToTAL sans perturber l'ensemble de ses pratiques existantes.

Étape 1 — Diagnostic de cycle (deux jours) : avant de changer quoi que ce soit, l'équipe réalise un diagnostic honnête de son cycle actuel en utilisant la Grille d'auto-évaluation SDLC ToTAL (voir Annexe C). Ce diagnostic identifie les capacités les plus déficientes et les plus urgentes à renforcer. Il produit un tableau de bord de maturité initial — la ligne de base contre laquelle les progrès seront mesurés.

Étape 2 — Choisir sa porte d'entrée (une semaine) : à partir du diagnostic, l'équipe choisit la capacité SDLC ToTAL par laquelle elle va commencer. En général, la porte d'entrée recommandée est SENSE — parce que c'est là que les problèmes de qualité ont leur source. Mais dans certains contextes (équipe qui a déjà une bonne pratique d'élicitation mais des spécifications fragiles), la porte d'entrée peut être SPECIFY. Le principe est de commencer par la capacité où le rapport bénéfice/effort est le plus favorable.

Étape 3 — Premier cycle ToTAL (quatre à six semaines) : l'équipe applique les pratiques ToTAL à sa capacité choisie pendant un cycle complet, en mesurant les indicateurs de santé définis. Elle tient un journal de bord des observations et des ajustements. À la fin du cycle, elle réalise une Rétrospective de Cycle Enrichie pour identifier les apprentissages et décider de la prochaine capacité à renforcer.

Étape 4 — Extension progressive (cycles suivants) : au rythme d'une capacité par cycle environ, l'équipe étend progressivement sa pratique ToTAL à toutes les phases du cycle. L'extension est organique : on ne force pas l'adoption de FORGE avant d'avoir solidifié SENSE et SPECIFY. La progression suit la logique de la chaîne Clarté → Qualité → Confiance.

Choisir sa porte d'entrée : SENSE, SPECIFY ou DELIVERY ?

🔭 **Repères — Porte d'entrée recommandée selon le contexte**

Symptôme dominant	Porte d'entrée recommandée	Première pratique à adopter
L'équipe construit souvent des fonctionnalités qui ne correspondent pas aux vrais besoins	SENSE	Atelier d'élicitation ToTAL + grille de qualification
Les user stories sont floues et génèrent des malentendus lors du développement	SPECIFY	Format SI/ALORS + matrice de traçabilité
Les livraisons génèrent fréquemment des incidents ou des retours négatifs	DELIVERY	Critères de sortie + contrat qualité de livraison
L'équipe répète les mêmes erreurs de cycle en cycle	CARE	

Rétrospective de Cycle Enrichie + Mémo d'Alimentation SENSE

Le code est difficile à maintenir et les bugs augmentent

FORGE

TDD + revue de code ToTAL + indicateurs CI

Indicateurs de succès à 30, 60, 90 jours

Horizon	Indicateurs de succès attendus	Signal d'alerte
30 jours	La grille de qualification SENSE est utilisée systématiquement. Les membres de l'équipe peuvent expliquer le format SI/ALORS. Le premier Lexique Vivant ToTAL contient au moins vingt termes définis. Un premier contrat qualité de livraison a été produit.	L'équipe continue à rédiger des user stories sans critères d'acceptation testables. Les pratiques ToTAL sont perçues comme « du travail en plus ».
60 jours	La matrice de traçabilité est maintenue à jour. Le pipeline CI inclut des indicateurs de santé ToTAL. Le taux de retravailler a diminué de façon mesurable. La première Rétrospective de Cycle Enrichie a produit un Mémo d'Alimentation SENSE.	Les métriques ToTAL ne sont produites que pour le reporting, pas utilisées par l'équipe. La couverture de tests stagne ou diminue.
90 jours	L'équipe peut mesurer son taux de qualification des besoins, sa couverture spécifications-tests, et son % d'apprentissages CARE intégrés au SENSE suivant. Les parties prenantes remarquent une amélioration de la pertinence des livraisons. La confiance dans les délais de livraison s'améliore.	Les indicateurs de santé ToTAL n'ont pas évolué depuis 30 jours. L'équipe ne peut pas identifier trois apprentissages concrets issus des 90 premiers jours de pratique ToTAL.

Phrase repère : *On ne commence pas ToTAL — on y entre. Et une fois entré, on ne revient pas en arrière, parce qu'on ne peut plus ignorer ce qu'on a appris à voir.*

ÉPILOGUE

Le cycle vivant

Quand le SDLC devient une discipline de maîtrise

Il y a un moment, dans l'histoire de chaque équipe qui pratique le SDLC ToTAL avec sérieux et continuité, où quelque chose change. Ce changement ne se produit pas lors d'un atelier ou d'une formation. Il ne s'annonce pas. Il se manifeste un jour, discrètement, dans une réunion de SENSE ordinaire : quelqu'un pose une question que personne n'avait posée avant, et tout le monde comprend immédiatement pourquoi cette question est fondamentale. Ou dans une revue de code : un développeur remarque une incohérence subtile entre la spécification et l'implémentation — une incohérence qui, dans le passé, serait passée inaperçue jusqu'à la production. Ou dans une rétrospective de cycle : l'équipe discute non seulement de ce qui s'est passé, mais de ce que cela signifie pour le cycle suivant.

Ce changement, c'est le moment où le cycle cesse d'être un processus et devient une discipline. La différence est profonde. Un processus est quelque chose qu'on suit. Une discipline est quelque chose qu'on pratique — quelque chose qui, à force de pratique, transforme non seulement ce qu'on fait mais qui on est en tant qu'équipe. Une équipe qui a atteint ce stade dans sa pratique ToTAL ne cherche plus à « appliquer la méthode » : elle pense naturellement en termes de clarté, de qualité et de confiance. Elle remarque instinctivement les besoins non qualifiés, les spécifications floues, les tests manquants. Elle traite chaque livraison comme une promesse à tenir et chaque phase CARE comme une source d'apprentissage précieux.

Cette transformation ne se produit pas en un mois. Elle prend généralement trois à six cycles complets — six à douze mois pour la plupart des équipes. Elle exige de la constance, de la réflexivité et une volonté de regarder honnêtement ce que le cycle révèle sur les pratiques de l'équipe. Elle exige aussi une posture non dogmatique : ToTAL est une discipline de maîtrise, pas un carcan. Il s'adapte aux contextes, respecte les contraintes, et accueille les innovations. Ce qui ne s'adapte pas, ce qui ne doit jamais s'adapter, c'est l'exigence fondamentale : chaque phase produit un apprentissage démontrable, pas seulement un livrable consommable.

La promesse du cycle ToTAL

Ce livre a commencé avec une promesse : chaque phase du SDLC ToTAL produit un apprentissage démontrable, pas seulement un livrable consommable. Rester dans cette promesse, cycle après cycle, c'est construire quelque chose qui dépasse la technique et la méthode : c'est construire une culture. Une culture dans laquelle la qualité est la norme, pas l'exception. Dans laquelle l'apprentissage est la mesure du progrès, pas la vélocité. Dans laquelle la confiance se construit sur des preuves, pas sur des espoirs.

DREAM-TEAM, au dernier cycle de la mission ORIGINALIUM, a produit un logiciel que les enseignants des vingt-deux écoles ont adopté spontanément et avec enthousiasme. Pas parce que l'équipe était plus talentueuse qu'elle ne l'était en semaine zéro. Mais parce qu'elle avait appris, cycle après cycle, à comprendre avant de construire, à spécifier avant de coder, à livrer avec des preuves, et à apprendre après chaque livraison. Ce logiciel n'était pas parfait. Mais il était juste — aligné sur de vrais besoins, construit avec une vraie qualité, livré avec une vraie confiance. Et c'est précisément cela, la promesse du cycle ToTAL.

> *« Le cycle vivant n'est pas celui qui tourne le plus vite — c'est celui qui apprend le mieux. Et un cycle qui apprend bien finit toujours par produire quelque chose dont l'équipe est fière et dont les utilisateurs ont vraiment besoin. C'est cela, la maîtrise. C'est cela, ToTAL. »*

> — Michel Bélanger, Collection ToTAL

Annexes

Annexe A — Gabarit de plan de qualité par phase SDLC ToTAL

Ce gabarit est à compléter au début de chaque phase du cycle. Il constitue l'engagement qualité de l'équipe pour la phase concernée et sert de base aux critères de sortie.

GABARIT DE PLAN DE QUALITÉ — PHASE : _____	
Projet / Mission	_____
Cycle #	_____
Date de début	_____
Date de fin prévue	_____
Responsable qualité de phase	_____

Section 1 — Objectifs qualité de la phase

(Quels objectifs qualité spécifiques cette phase doit-elle atteindre ? Maximum cinq objectifs.)

1. _____
2. _____
3. _____
4. _____

5. _____

Section 2 — Métriques de qualité de phase

Métrique	Valeur cible	Méthode de mesure	Fréquence
1. ___	___	___	___
2. ___	___	___	___
3. ___	___	___	___

Section 3 — Risques qualité identifiés

Risque	Probabilité (1-5)	Impact (1-5)	Mitigation	Propriétaire
___	___	___	___	___
___	___	___	___	___
___	___	___	___	___

Section 4 — Critères de sortie de phase

Critère	Comment vérifié	Qui valide	Satisfait ? (O/N)
___	___	___	___
___	___	___	___
___	___	___	___
___	___	___	___

Section 5 — Apprentissages de phase (à compléter en fin de phase)

Ce qui a bien fonctionné : _____

Ce qui doit être amélioré : _____

Ce qui sera intégré au prochain cycle : _____

Annexe B — Référentiel SI/ALORS : 25 exemples commentés

Les exemples suivants sont organisés par domaine et illustrent l'application du format SI/ALORS dans des contextes variés. Chaque exemple est accompagné d'un commentaire explicatif.

#	Domaine	Spécification SI/ALORS	Commentaire
1	Authentification	*SI un utilisateur saisit un mot de passe incorrect pour la troisième fois consécutive, ALORS son compte est verrouillé pour trente minutes et il reçoit un courriel d'alerte à l'adresse enregistrée.*	Condition précise (3e tentative consécutive), comportement double (verrouillage + courriel), mesure incluse (30 min). Testable automatiquement.
2	Performance	*SI un utilisateur charge la page d'accueil du portail avec une connexion standard (10 Mbps), ALORS la page est entièrement affichée en moins de deux secondes, mesurée par le Time to Interactive (TTI).*	Contexte précis (10 Mbps), métrique définie (TTI), seuil quantifié (2s). La référence à la métrique standard (TTI) assure la reproductibilité du test.
3	Gestion des erreurs	*SI un enseignant*	Message d'erreur

#	Domaine	Spécification SI/ALORS	Commentaire
		tente de soumettre un devoir avec une date d'échéance antérieure à la date d'aujourd'hui, ALORS le système affiche le message : « La date d'échéance doit être postérieure à la date du jour » et conserve toutes les données déjà saisies dans le formulaire.	exact spécifié (facilite les tests UI). Comportement supplémentaire important : conservation des données (qualité UX). Condition d'échec claire.
4	Accessibilité	*SI un utilisateur navigue dans l'application uniquement avec le clavier (sans souris), ALORS toutes les fonctionnalités disponibles à la souris sont accessibles via la touche Tab et les touches directionnelles, sans exception.*	Condition précise (clavier uniquement), comportement universel (toutes fonctionnalités), sans exception (zéro tolérance). Testable avec des outils d'accessibilité.
5	Notifications	*SI un administrateur publie une annonce marquée « urgente », ALORS tous les utilisateurs actifs du réseau reçoivent une notification push dans un délai inférieur à soixante secondes, indépendamment*	Segment précis (utilisateurs actifs), SLA défini (60s), comportement indépendant du contexte. Le terme « actif » doit être défini dans le Lexique Vivant.

#	Domaine	Spécification SI/ALORS	Commentaire
		de la page sur laquelle ils se trouvent.	
6	Données	*SI un utilisateur exporte un rapport en format CSV, ALORS le fichier exporté contient exactement les mêmes données que celles affichées à l'écran, dans le même ordre et sans troncature.*	Critère d'exactitude (mêmes données), critère d'ordre, critère d'intégrité (sans troncature). Trois vérifications dans une seule spec — acceptable car elles sont intrinsèquement liées.
7	Sécurité	*SI un utilisateur tente d'accéder à une ressource pour laquelle son rôle ne dispose pas des droits nécessaires, ALORS le système retourne un code HTTP 403 et affiche une page d'erreur explicite — sans révéler l'existence ou le contenu de la ressource.*	Code HTTP spécifié (403), comportement de sécurité critique (ne pas révéler l'existence). La nuance « sans révéler » est fondamentale pour la sécurité.
8	Internationalisation	*SI un utilisateur dont la langue d'interface est le français canadien saisit une date, ALORS le format attendu est JJ/MM/AAAA et toute saisie dans un autre format génère un message d'erreur formaté en français.*	Localisation précise (français canadien, pas juste « français »), format explicite (JJ/MM/AAAA), comportement d'erreur localisé. Le Lexique Vivant doit définir « langue d'interface ».

#	Domaine	Spécification SI/ALORS	Commentaire
9	Hors-ligne	*SI un élève accède à un cours préalablement téléchargé et que sa connexion réseau est interrompue, ALORS il peut lire l'intégralité du cours et soumettre ses réponses aux questions, qui seront synchronisées automatiquement dès le retour de la connexion.*	Condition double (cours téléchargé + coupure réseau), comportements multiples liés (lecture + soumission + synchronisation). La synchronisation doit avoir sa propre spec SI/ALORS complémentaire.
10	Recherche	*SI un utilisateur effectue une recherche avec un terme de deux caractères ou moins, ALORS le système affiche un message guidant l'utilisateur à saisir au moins trois caractères, sans lancer de recherche.*	Seuil précis (≤2 caractères), comportement explicite (message + pas de recherche), message guidant (qualité UX). Simple, clair, testable.
11	Rapports	*SI un directeur d'école génère le rapport de participation mensuel pour son école, ALORS le rapport inclut uniquement les données des élèves inscrits dans son école au premier jour du mois concerné.*	Périmètre précis (son école), règle métier explicite (inscrits au premier jour du mois). Cette règle métier doit être dans le Lexique Vivant sous « inscrits ».
12	Onboarding	*SI un nouvel utilisateur se*	Contrainte UX (5 étapes max), droit

#	Domaine	Spécification SI/ALORS	Commentaire
		connecte pour la première fois, ALORS il est guidé par une séquence d'introduction de cinq étapes maximum qu'il peut interrompre à tout moment et reprendre ultérieurement.	de l'utilisateur (interrompre), persistance (reprendre ultérieurement). Trois critères de qualité UX dans une spec.
13	Intégration	*SI le système de gestion scolaire externe envoie une mise à jour de la liste des élèves, ALORS le système intègre cette mise à jour en moins de cinq minutes et enregistre un log horodaté de l'opération.*	Source externe définie, SLA d'intégration (5 min), traçabilité obligatoire (log horodaté). Le log est une exigence de qualité et d'audit.
14	Suppression	*SI un utilisateur supprime un compte élève, ALORS le système affiche une confirmation avec le nombre d'éléments qui seront supprimés et demande une confirmation explicite par saisie du mot « SUPPRIMER ».*	Pattern de confirmation sécurisé (saisie du mot), information préalable (nombre d'éléments), protection contre la suppression accidentelle. Testable UI.
15	Pagination	*SI une liste contient plus de cinquante éléments, ALORS elle est paginée automatiquement avec vingt-cinq éléments par*	Seuil de déclenchement (>50), taille de page (25), éléments de navigation explicitement listés. Détail suffisant pour l'implémentation, pas au-delà.

#	Domaine	Spécification SI/ALORS	Commentaire
		page et inclut une navigation explicite (Première, Précédente, Numéros de page, Suivante, Dernière).	
16	Audit	*SI un administrateur modifie le rôle d'un utilisateur, ALORS l'action est enregistrée dans le journal d'audit avec : l'identifiant de l'administrateur, l'identifiant de l'utilisateur modifié, l'ancien rôle, le nouveau rôle et l'horodatage UTC.*	Cinq éléments de log explicitement listés. La précision UTC évite les ambiguïtés de fuseau horaire dans les systèmes distribués.
17	Téléversement	*SI un enseignant tente de téléverser un fichier de plus de cinquante mégaoctets, ALORS le système refuse le téléversement avant son début (pas après) et affiche un message précisant la limite de taille.*	Limite précise (50 Mo), timing critique (avant, pas après — économie de bande passante), message informatif. Le « avant » est une décision de qualité technique et UX.
18	Temps réel	*SI deux enseignants modifient simultanément le même cours, ALORS le système informe chacun de la présence de*	Scénario concurrent (simultané), SLA de notification (3s), garantie d'intégrité (sans écrasement). La méthode de résolution des conflits doit être spécifiée

#	Domaine	Spécification SI/ALORS	Commentaire
		l'autre en temps réel (délai maximal de trois secondes) et prévient tout écrasement de données.	séparément.
19	Archivage	*SI une année scolaire est clôturée par un administrateur, ALORS toutes les données de l'année sont archivées et restent accessibles en lecture seule pendant sept ans, conformément aux exigences légales applicables.*	Durée légale précise (7 ans), nature de l'accès (lecture seule), déclencheur précis (clôture par admin). La conformité légale doit être validée par les parties prenantes juridiques.
20	Tableau de bord	*SI un directeur d'école ouvre son tableau de bord, ALORS les indicateurs affichés reflètent les données des vingt-quatre dernières heures et sont mis à jour automatiquement toutes les heures sans rechargement manuel de la page.*	Fenêtre temporelle (24h), fréquence de mise à jour (1h), comportement UX (sans rechargement). Trois critères mesurables dans une spec liée.
21	Multi-langues	*SI un utilisateur change sa langue d'interface dans ses préférences, ALORS l'ensemble de l'interface bascule dans la nouvelle langue*	SLA (2s), sans déconnexion (continuité de session), sans perte (intégrité des données). Trois garanties qualité explicites.

#	Domaine	Spécification SI/ALORS	Commentaire
		dans les deux secondes suivant la sauvegarde, sans déconnexion et sans perte des données en cours de saisie.	
22	Impression	*SI un enseignant imprime un rapport de progression, ALORS la version imprimée contient exactement les mêmes données que la version affichée, formatées sur un maximum de deux pages A4 en orientation portrait.*	Cohérence données (même données), contrainte de format (2 pages A4 portrait). Simple et testable avec un aperçu avant impression.
23	Inactivité	*SI une session utilisateur est inactive depuis trente minutes, ALORS le système affiche un avertissement avec un compte à rebours de soixante secondes avant la déconnexion automatique, en sauvegardant automatiquement tout formulaire en cours.*	Délai d'inactivité (30 min), avertissement (60s de compte à rebours), sauvegarde automatique (protection des données utilisateur). Pattern de sécurité et de qualité UX combinés.
24	API externe	*SI le service externe de messagerie est indisponible lors de l'envoi d'une notification, ALORS le système*	Comportement de résilience complet : mise en file d'attente, intervalle de retry (5 min), durée maximale (24h), escalade (notification admin).

#	Domaine	Spécification SI/ALORS	Commentaire
		met la notification en file d'attente et retente l'envoi toutes les cinq minutes pendant un maximum de vingt-quatre heures, puis notifie l'administrateur en cas d'échec définitif.	Qualité de service opérationnelle spécifiée.
25	Consentement	*SI un élève mineur (moins de seize ans) s'inscrit à la plateforme, ALORS le système requiert le courriel d'un parent ou tuteur légal pour envoyer une demande de consentement, et l'accès est suspendu jusqu'à la réception de ce consentement.*	Seuil légal précis (16 ans), mécanisme de consentement (courriel parent), accès conditionnel. Conformité légale et privacy-by-design. Doit être validé par les parties juridiques.

Annexe C — Grille d'auto-évaluation SDLC ToTAL (par phase, 5 niveaux)

Cette grille permet à une équipe d'évaluer sa maturité dans chacune des six capacités du SDLC ToTAL. Pour chaque capacité, évaluez votre équipe sur une échelle de 1 à 5 selon les descripteurs ci-dessous. La somme par capacité donne un score de maturité (max 25 par capacité, max 150 au total).

Légende des niveaux de maturité

Niveau	Nom	Description générale
1	**Initial**	L'activité est réalisée de façon ad hoc, sans méthode ni mesure.
2	**Conscient**	L'équipe reconnaît l'importance de l'activité mais ne la pratique pas systématiquement.
3	**Défini**	L'activité est définie, documentée et pratiquée par la majorité de l'équipe.
4	**Géré**	L'activité est mesurée, et les résultats sont utilisés pour l'amélioration continue.
5	**Optimisé**	L'activité est intégrée naturellement dans la culture de l'équipe et s'améliore de façon continue et proactive.

SENSE — Grille d'auto-évaluation

Critère	1 Initial	2 Conscient	3 Défini	4 Géré	5 Optimisé	Score
Élicitation des besoins	Besoins tirés de documents sans consultation des utilisateurs	Quelques entretiens utilisateurs, non structurés	Atelier d'élicitation ToTAL utilisé systématiquement	Résultats des ateliers mesurés et améliorés	L'élicitation évolue avec les apprentissages CARE de chaque cycle	___
Qualification des besoins	Aucune qualification formelle	Qualification informelle, non documentée	Grille des 12 critères utilisée et documentée	Taux de qualification mesuré et suivi	La grille est affinée en fonction des apprentissages de cycle	___

Critère	1 Initial	2 Conscient	3 Défini	4 Géré	5 Optimisé	Score
Lexique Vivant	Aucun glossaire	Glossaire informel, non maintenu	Lexique Vivant ToTAL maintenu et accessible	Utilisation du lexique mesurée (% de specs référençant le lexique)	Le lexique est une ressource organisationnelle multi-équipes	___
Indicateurs de mesure des besoins	Aucun indicateur défini	Indicateurs mentionnés mais non formalisés	Chaque besoin qualifié a au moins un indicateur de mesure	Les indicateurs alimentent directement CARE	Les indicateurs évoluent avec les apprentissages de l'usage réel	___
Implication des utilisateurs réels	Aucun utilisateur final consulté	Utilisateurs consultés occasionnellement	Au moins un utilisateur final présent dans chaque atelier SENSE	Diversité des profils utilisateurs mesurée et assurée	Les utilisateurs sont des partenaires actifs de SENSE	___
TOTAL SENSE	___/25					

SPECIFY — Grille d'auto-évaluation

Critère	1	2	3	4	5	Score
Format SI/ALORS	User stories sans critères d'acceptation	Critères d'acceptation informels	Format SI/ALORS utilisé systématiquement	Qualité des SI/ALORS mesurée (pathologies identifiées)	Le format évolue avec les apprentissages de SPECIFY	___
Traçabilité besoin-spec-test	Aucune traçabilité	Traçabilité partielle et	Matrice de traçabilité	Taux de couverture mesuré	La traçabilité est	___

Critère	1	2	3	4	5	Score
		informelle	complète et maintenue	et cible définie	automatisée et temps réel	
Validation par les parties prenantes	Specs rédigées sans validation externe	Validation informelle par le PO seulement	Validation formelle PO + utilisateurs représentatifs	Taux de specs validées mesuré, suivi, objectif défini	La validation est un processus collaboratif continu	___
Gestion des pathologies de specs	Pathologies non identifiées	Pathologies identifiées ponctuellement	Revue de specs systématique avec grille de pathologies	Fréquence des pathologies mesurée par type	Les pathologies récurrentes sont éliminées proactivement	___
Vivance des spécifications	Specs figées, jamais mises à jour	Mises à jour occasionnelles et informelles	Processus de mise à jour documenté et respecté	Historique des modifications tracé et auditable	Les specs évoluent en temps réel avec les apprentissages	___
TOTAL SPECIFY	___/25					

PLAN — Grille d'auto-évaluation

Critère	1	2	3	4	5	Score
Intégration de la qualité dans le plan	Plan = effort et délais uniquement	Quelques critères qualité mentionnés	Canevas ToTAL complet, qualité intégrée	Risques qualité mesurés et suivis	La qualité est la dimension première du plan	___
Critères de sortie de phase	Aucun critère de sortie défini	Critères informels et variables	Critères définis avant chaque phase	Critères mesurés et taux de satisfaction suivi	Les critères évoluent et s'améliorent de cycle en cycle	___

Critère	1	2	3	4	5	Score
RACI qualité	Responsabilités qualité implicites	RACI général, pas spécifique à la qualité	RACI qualité ToTAL documenté et compris	Respect du RACI mesuré	Le RACI est une référence culturelle, pas un document	___
TOTAL PLAN	___/15					

FORGE — Grille d'auto-évaluation

Critère	1	2	3	4	5	Score
TDD / BDD	Tests écrits après le code, ou pas du tout	Tests écrits après, mais systématiquement	TDD/BDD pratiqué par la majorité de l'équipe	Couverture mesurée, objectif cible tenu	Les specs SI/ALORS génèrent automatiquement des tests	___
Revue de code qualité	Revues inexistantes ou formelles seulement	Revues réalisées, non structurées	Revues ToTAL sur 4 axes, systématiques	Qualité des revues mesurée (défauts détectés par revue)	Les revues sont un moment de formation et d'amélioration continue	___
Intégration continue (CI)	Pas de CI, intégration manuelle	CI en place, résultats peu utilisés	Pipeline CI avec indicateurs de santé ToTAL	Pipeline rouge = priorité absolue, respectée systématiquement	Le CI est un outil de documentation qualité temps réel	___
Gestion de la dette technique	Dette non reconnue ou ignorée	Dette reconnue mais non documentée	Dette délibérée documentée avec plan de remboursement	Dette mesurée (outils statiques), objectif de réduction défini	La dette est un indicateur de santé suivi au même titre que la vélocité	___
Conformit	Aucune	Vérificatio	Conformit	Taux de	La	___

Critère	1	2	3	4	5	Score
é specs-implémentation	vérification de conformité	n informelle par le développeur	é vérifiée systématiquement en revue de code	conformité mesuré et suivi	conformité est automatiquement vérifiée par les tests BDD	
TOTAL FORGE	___/25					

DELIVERY & CARE — Grille d'auto-évaluation

Critère	1	2	3	4	5	Score
Contrat qualité de livraison	Livraison = déploiement réussi, sans autre critère	Quelques vérifications avant livraison, informelles	Contrat qualité ToTAL produit pour chaque livraison	Taux de satisfaction des critères mesuré	La livraison est une déclaration qualité, pas une opération technique	___
Métriques post-livraison (CARE)	Aucune métrique post-livraison	Métriques génériques, non actionnables	Métriques de signaux utiles définies et collectées	Métriques utilisées pour des décisions d'intervention	Les métriques évoluent avec les apprentissages de CARE	___
Boucle CARE → SENSE	Aucune connexion entre CARE et le cycle suivant	Rétrospective classique sans connexion explicite au SENSE suivant	Rétrospective de Cycle Enrichie + Mémo d'Alimentation SENSE	% d'apprentissages CARE intégrés au SENSE suivant mesuré	La boucle est naturelle et automatique dans la culture de l'équipe	___
Mémoire organisationnelle	Aucune documentation des apprentissages	Notes de rétrospective, non indexées ni accessibles	Mémoire ToTAL maintenue (Lexique, Mémos, Métriques)	Mémoire utilisée activement au démarrage de chaque SENSE	La mémoire est une ressource multi-équipes et multi-projets	___
TOTAL	___/20					

Critère	1	2	3	4	5	Score
DELIVERY + CARE						

Interprétation du score global

Score total (sur 110)	Niveau de maturité	Priorité d'action
22 — 44	**Initial**	Commencer par SENSE. Atelier d'élicitation ToTAL comme première pratique.
45 — 66	**Conscient**	Formaliser les pratiques existantes. Introduire le format SI/ALORS et la matrice de traçabilité.
67 — 88	**Défini**	Mesurer systématiquement. Introduire les indicateurs de santé ToTAL et la boucle CARE → SENSE.
89 — 100	**Géré**	Utiliser les métriques pour amélioration proactive. Partager les apprentissages avec d'autres équipes.
101 — 110	**Optimisé**	Contribuer à l'évolution de la discipline ToTAL. Mentor d'autres équipes.

Annexe D — Glossaire ToTAL SDLC

Ce glossaire définit les termes fondamentaux du SDLC ToTAL dans l'ordre alphabétique. Il constitue le noyau du Lexique Vivant ToTAL pour les équipes qui démarrent avec la discipline. Chaque équipe est invitée à enrichir ce glossaire avec les termes spécifiques à son domaine métier.

Terme	Définition ToTAL	Distinguer de
Apprentissage de phase	Connaissance nouvelle acquise par l'équipe au cours d'une phase du SDLC ToTAL, portant sur le besoin, le produit, le processus ou le contexte. Distinct du livrable de phase.	Livrable de phase (ce qui est transmis à la phase suivante)
BEASTEST	Cadre d'automatisation des tests ToTAL (Build, Execute, Assess, Store, Track, Evolve, Systematize, Transfer). Principes gouvernant la construction, la maintenance et l'utilisation des tests automatisés dans le SDLC ToTAL.	Un outil spécifique (BEASTEST est un cadre de principes, pas un outil)
CARE	Sixième capacité du SDLC ToTAL. Phase d'observation active, de mesure et d'apprentissage à partir de l'usage réel d'une livraison. Structure : Diffuser, Veiller, Intervenir, Recueillir.	Maintenance (CARE est proactif et orienté apprentissage ; la maintenance est réactive et orientée correction)
Chaîne Clarté → Qualité → Confiance	Principe causal fondateur de ToTAL. La clarté (compréhension partagée et précise) est la condition préalable à la qualité (conformité vérifiable). La qualité est la fondation de la confiance (capacité à compter sur ce que le cycle produit).	Slogan (c'est un modèle causal, pas un slogan)
Contrat qualité de livraison	Artefact d'une page documentant, pour chaque livraison DELIVERY : le périmètre livré, les preuves de qualité disponibles, et la décision de livraison (qui, sur la base de quoi, avec quelles réserves).	Note de release (centrée sur le contenu) ; release notes (destinées aux utilisateurs)
Critères de sortie de phase	Ensemble de conditions qui doivent être satisfaites avant qu'une phase du SDLC ToTAL soit considérée comme terminée et avant que la phase suivante commence. Distinct de la « définition de done » des	Définition de done Scrum (porte sur les stories, pas sur les phases)

Terme	Définition ToTAL	Distinguer de
	user stories.	
Cycle vivant	SDLC ToTAL dans son état de maturité optimal : un cycle qui produit des apprentissages démontrables à chaque phase, qui alimente sa propre amélioration, et qui traduit la chaîne Clarté → Qualité → Confiance en résultats tangibles et mesurables.	Cycle opérationnel (séquence d'étapes exécutées sans mécanisme d'apprentissage intégré)
Dette conceptuelle	Coût différé des concepts mal définis ou non définis dans les spécifications. Se manifeste par des malentendus de développement, des incohérences d'implémentation, et des correctifs tardifs dus à des ambiguïtés de vocabulaire.	Dette technique (porte sur le code ; la dette conceptuelle porte sur les mots et les définitions)
DELIVERY	Cinquième capacité du SDLC ToTAL. Acte de mettre ce qui a été construit entre les mains de ses destinataires, avec les preuves de qualité qui démontrent que ce qui est livré est digne de confiance.	Déploiement (opération technique ; DELIVERY est un acte social et professionnel de qualité)
DREAM-TEAM	Équipe fictive utilisée comme fil conducteur narratif dans la collection ToTAL. Composée de sept membres aux rôles variés, engagée dans la mission ORIGINALIUM. Sert à illustrer l'application concrète des pratiques ToTAL dans des situations réalistes.	Équipe modèle (DREAM-TEAM fait des erreurs et apprend — elle n'est pas présentée comme parfaite)
FORGE	Quatrième capacité du SDLC ToTAL. Phase de construction où la clarté produite par SENSE et SPECIFY est transformée en logiciel fonctionnel avec qualité intégrée — par le TDD, le BDD, les revues de code et l'intégration	Développement (terme générique ; FORGE implique une posture spécifique de qualité bâtie, pas contrôlée)

Terme	Définition ToTAL	Distinguer de
	continue.	
Grille de qualification des besoins	Outil ToTAL de douze critères organisés en quatre dimensions (Pertinence, Précision, Testabilité, Traçabilité) permettant de vérifier qu'un besoin est prêt à être spécifié. Score de 1 à 5 par critère ; seuil de passage : moyenne ≥ 3 par dimension.	Définition de ready (Scrum) — plus générique et moins structurée
Indicateur de santé du cycle	Métrique opérationnelle que l'équipe utilise elle-même pour évaluer si chaque phase du SDLC ToTAL produit les apprentissages et la qualité attendus. Distinct des métriques de reporting.	KPI de reporting (destiné au management ; l'indicateur de santé est destiné à l'équipe)
Lexique Vivant ToTAL	Glossaire collaboratif maintenu par l'équipe, définissant les termes clés du domaine métier utilisés dans les spécifications. Outil central de lutte contre la dette conceptuelle. Mis à jour en continu, révisé à chaque début de SENSE.	Dictionnaire (statique, généraliste) ; Ubiquitous Language DDD (similaire, mais ToTAL y intègre les dimensions de qualification et de traçabilité)
Matrice de traçabilité	Artefact de gouvernance de SPECIFY qui relie chaque besoin qualifié (SENSE) à ses spécifications SI/ALORS (SPECIFY) et à ses tests automatisés (FORGE). Outil de vérification de la couverture et de l'intégrité du cycle.	Matrice de couverture de tests (ne remonte pas jusqu'aux besoins ; la matrice ToTAL couvre toute la chaîne)
Mémo d'Alimentation SENSE	Artefact d'une à deux pages produit lors de la Rétrospective de Cycle Enrichie, capturant les décisions issues des apprentissages CARE qui modifieront l'approche SENSE du cycle suivant. Principal mécanisme de connexion entre CARE et SENSE.	Action item de rétrospective (ponctuel, non structuré) ; le Mémo est un engagement structuré et traçable
Mémoire	Ensemble des	Documentation projet

Terme	Définition ToTAL	Distinguer de
organisationnelle ToTAL	apprentissages documentés, qualifiés et indexés, produits par tous les cycles passés d'une équipe. Composée du Lexique Vivant, des Mémos d'Alimentation SENSE, des registres des risques qualité et des rapports de métriques post-livraison.	(archive statique ; la mémoire ToTAL est un système actif d'alimentation des cycles futurs)
ORIGINALIUM	Mission fictive utilisée comme fil conducteur narratif dans la collection ToTAL. Projet de développement d'une plateforme de gestion de contenu éducatif pour un réseau de vingt-deux écoles primaires, conduite par l'équipe DREAM-TEAM.	Étude de cas réelle (ORIGINALIUM est fictif mais conçu pour illustrer des situations typiques réalistes)
PLAN	Troisième capacité du SDLC ToTAL. Acte de maîtrise par lequel l'équipe décide avec lucidité ce qu'elle va construire, dans quel ordre, avec quels critères de qualité, avec quelle gestion des risques, et avec quels critères de sortie.	Planification de sprint Scrum (plus restreinte ; PLAN ToTAL couvre l'ensemble du cycle et intègre explicitement la qualité comme dimension primaire)
Preuve de qualité de phase	Ensemble d'indicateurs, de livrables et de démonstrations qui confirment non seulement qu'une phase a été exécutée, mais que ce qu'elle devait permettre de comprendre ou de produire a effectivement été accompli avec la qualité requise.	Livrable de phase (ce qui est transmis ; la preuve de qualité valide la valeur de ce livrable)
RACI qualité ToTAL	Déclinaison du modèle RACI (Responsible, Accountable, Consulted, Informed) appliquée spécifiquement aux preuves de qualité de chaque phase du SDLC ToTAL. Clarifie qui produit, qui valide, qui est consulté et qui est informé pour chaque preuve de qualité.	RACI de projet (porte sur les livrables généraux ; le RACI qualité ToTAL porte exclusivement sur les preuves de qualité)

Terme	Définition ToTAL	Distinguer de
Rétrospective de Cycle Enrichie	Cérémonie ToTAL se tenant en fin de cycle complet (et non en fin de sprint). Elle examine les apprentissages sur le produit, sur le cycle et sur les six capacités SDLC ToTAL, et produit un Mémo d'Alimentation SENSE.	Rétrospective de sprint Scrum (porte sur le processus du sprint ; la Rétrospective de Cycle Enrichie porte sur l'ensemble du cycle et ses apprentissages produit)
SDLC ToTAL	Software Development Life Cycle ToTAL. Cadre de cycle de développement logiciel organisé autour de six capacités (SENSE, SPECIFY, PLAN, FORGE, DELIVERY, CARE) et d'un principe fondateur : chaque phase produit un apprentissage démontrable, pas seulement un livrable consommable.	SDLC classique (séquence d'étapes à franchir ; le SDLC ToTAL est un système apprenant)
SENSE	Première capacité du SDLC ToTAL. Processus structuré de compréhension du vrai besoin avant toute spécification ou construction. Quatre gestes : écouter, nommer, qualifier, mesurer.	Recueil d'exigences (passif, centré sur la demande formulée ; SENSE est actif, centré sur le besoin réel)
SPECIFY	Deuxième capacité du SDLC ToTAL. Traduction des besoins qualifiés issus de SENSE en spécifications vivantes au format SI/ALORS — précises, testables, mesurables et traçables.	Rédaction de user stories (souvent sans critères d'acceptation formalisés ; SPECIFY exige des critères testables et traçables)
Spécification vivante	Spécification rédigée au format SI/ALORS, maintenue à jour avec l'évolution du besoin, directement liée à un test automatisé qui la vérifie. Possède cinq propriétés : précise, testable, traçable, mesurable, vivante.	Document de spécification statique (figé à sa date de rédaction ; la spécification vivante évolue avec le besoin)
Système apprenant	Dans le contexte ToTAL, système de développement logiciel optimisé non seulement pour produire	Système d'exécution (optimisé pour la production de sorties à partir d'entrées ; sans

Terme	Définition ToTAL	Distinguer de
	des sorties (livrables), mais aussi pour améliorer sa propre capacité à produire de meilleures sorties en fonction de ce qu'il observe, mesure et comprend. Le SDLC ToTAL est conçu comme un système apprenant.	mécanisme d'apprentissage intégré)
ToTAL	Collection de quatorze livres sur l'art de réaliser un produit logiciel. Acronyme non développé, désignant une approche de maîtrise holistique du développement logiciel, non dogmatique, centrée sur la chaîne Clarté → Qualité → Confiance.	Cadre agile (ToTAL est une discipline de maîtrise qui s'intègre aux cadres existants, pas un cadre concurrent)

Collection ToTAL — L'art de réaliser un produit logiciel

Fondations — Livre 2 sur 14

Le SDLC ToTAL — SENSE · SPECIFY · PLAN · FORGE · DELIVERY · CARE

Le cycle vivant de la maîtrise logicielle

Auteur Michel Bélanger **Crédits visuels** Soléane Bélanger **Date de publication** Septembre 2026

Les livres de la collection ToTAL Livre 1 — Les fondations de la qualité logicielle **Livre 2 — Le SDLC ToTAL (ce volume)** Livre 3 — SENSE : L'art de comprendre Livre 4 — SPECIFY : Les spécifications vivantes Livre 5 — PLAN : Décider avec lucidité Livre 6 — FORGE : Construire avec qualité Livre 7 — DELIVERY : Livrer avec confiance Livre 8 — CARE : L'apprentissage permanent Livres 9 à 14 — Applications et contextes avancés

---

« La maîtrise n'est pas un état — c'est une trajectoire. Chaque cycle bien pratiqué vous rapproche un peu plus de ce que vous voulez construire : non seulement un logiciel de qualité, mais une équipe qui sait pourquoi et comment elle produit de la qualité. »

— Michel Bélanger

Le pont vers ToTAL

— et vers tout ce qui reste à découvrir —

Ce que ToTAL apporte au SDLC

Il serait tentant, après avoir lu l'histoire du SDLC dans toute sa complexité, ses fragmentations successives et sa réconciliation progressive, de conclure que ToTAL est une réponse de plus à une question déjà trop encombrée de réponses. Ce serait se méprendre sur ce que ToTAL apporte réellement. ToTAL n'apporte pas un nouveau SDLC. Il apporte les noms et la structure que le SDLC réconcilié attendait depuis toujours — ces noms précis, stables, opérationnels, qui permettent enfin de parler du cycle complet sans qu'on se perde dans le vocabulaire de tel ou tel cadre, de telle ou telle école de pensée, de telle ou telle époque.

Le premier apport concret de ToTAL, celui qui illumine immédiatement la structure du cycle réconcilié, est la nomination de ses deux hémisphères fondamentaux : **SPECIFY** et **DELIVERY**. Ces deux hémisphères n'ont pas été inventés par ToTAL — ils ont toujours existé dans tout cycle complet, comme les deux moitiés d'un cerveau qui ne peuvent fonctionner séparément sans perdre une part essentielle de leur intelligence. Ils avaient simplement été mal nommés, mal délimités, mal articulés. ToTAL les nomme avec précision. SPECIFY — que l'on peut désigner par l'abréviation cs, pour *continuous specification* — couvre les phases ÉLICITER, DÉFINIR et PLANIFIER. C'est

l'hémisphère du sens, de la clarté, de la décision. C'est là que le logiciel est pensé avant d'être construit, que les besoins sont entendus avant d'être traduits, que les décisions sont prises avant que la construction commence. DELIVERY — ci/cd, pour *continuous integration et continuous delivery* dans leur sens le plus complet — couvre les phases IMPLÉMENTER, DIFFUSER et CONFIRMER. C'est l'hémisphère de la construction, de la livraison, de la confirmation. C'est là que ce qui a été pensé prend forme, voyage vers les utilisateurs, et est finalement confirmé comme correspondant ou non à ce qui était attendu.

Ce qui rend cette nomination transformatrice n'est pas sa nouveauté intellectuelle — c'est sa clarté opérationnelle. Des équipes qui se disputaient depuis des années sur la frontière entre « produit » et « livraison », entre « analyse » et « développement », entre « recette » et « production » trouvent dans SPECIFY et DELIVERY des balises stables. Pas des cases à remplir. Des territoires à habiter avec cohérence, en sachant exactement ce que l'on cherche à accomplir dans chacun, et ce que l'on risque si l'on le court-circuite.

Le deuxième apport de ToTAL est tout aussi structurant : la nomination des trois grandes sections que ToTAL désigne sous le nom de ModDevOps — **SENSE**, **FORGE** et **CARE**. DevOps avait fait un pas décisif en réconciliant Dev et Ops — en affirmant que le développement et l'opérationnel ne pouvaient plus se regarder de part et d'autre d'un mur organisationnel sans que la qualité en souffre. C'était juste. C'était nécessaire. Mais c'était incomplet. Il manquait une troisième dimension : la modélisation, le sens, le design — tout ce que ToTAL nomme **SENSE**, ou Mod dans l'acronyme ModDevOps. SENSE couvre les phases ÉLICITER et DÉFINIR. C'est le territoire de la compréhension profonde avant l'action : écouter les besoins, les qualifier, les nommer avec précision, les modéliser sous une forme qui permettra à la construction de ne pas s'égarer. FORGE — Dev dans ModDevOps — couvre les phases PLANIFIER et IMPLÉMENTER. C'est le territoire de la construction intentionnelle : planifier avec lucidité, construire avec rigueur, intégrer avec cohérence. Et CARE — Ops dans ModDevOps — couvre les phases DIFFUSER et CONFIRMER. C'est le territoire de la responsabilité continue : livrer avec confiance, veiller sur ce qui est déployé, confirmer que la valeur promise est bien la valeur livrée, et nourrir le cycle suivant de ce que ce territoire a appris.

À retenir :

Software Development Life Cycle. Development. Réalisation. Pas juste le code. Pas juste l'implémentation. La réalisation complète d'un produit logiciel — de l'idée à la confirmation de sa valeur. C'était là depuis le début. ToTAL ne fait que lui rendre son nom entier, son sens entier, sa cohérence entière.

Le troisième apport de ToTAL est peut-être le plus libérateur, parce qu'il met fin à une confusion qui a coûté cher à l'industrie pendant des décennies : la confusion entre le SDLC comme cadre personnalisable et le SDLC comme discipline universelle. Pendant trop longtemps, l'idée s'est imposée qu'une organisation pouvait — et même devait — « choisir son SDLC » comme on choisit un outil dans une boîte à outils. Certains choisissaient Scrum. D'autres choisissaient Kanban. D'autres encore inventaient un hybride maison, convaincu que leur contexte était si unique qu'aucune approche standard ne pouvait leur convenir. Cette liberté apparente a produit une fragmentation réelle : des équipes qui ne parlent pas le même langage, des organisations où chaque département a son propre cycle, des programmes entiers où personne ne peut dire avec clarté à quelle étape du cycle on se trouve et ce que cela signifie. ToTAL y met fin — non par dogmatisme, mais par cohérence. Il ne dit pas : « suivez le SDLC ToTAL comme s'il était une marque ou une méthode propriétaire. » Il dit simplement : voici ce que le SDLC a toujours été, nommé avec précision, structuré avec cohérence. C'est le SDLC. Un seul. Complet. Réconcilié. Et dans ce SDLC réconcilié, la word « Développement » retrouve enfin son sens initial et complet — non pas le codage, non pas l'implémentation technique, mais la réalisation dans toute sa plénitude : de la conception à la confirmation, de l'idée à la valeur livrée.

Ce n'est plus un SDLC custom selon le goût de l'entreprise, selon le profil de son équipe, selon les convictions de son dernier consultant. Ce n'est pas non plus une déclaration que « le SDLC ToTAL est le meilleur et il faut le suivre ». Ce n'est pas davantage l'argument — entendu de plus en plus fréquemment avec l'essor de l'IA générative — que les cycles de développement sont désormais obsolètes, que l'IA permet de s'en affranchir, que la livraison continue rend la structure du cycle inutile.

C'est le contraire exact : jamais le SDLC n'a été aussi nécessaire qu'à l'ère de l'IA, précisément parce que l'IA amplifie tout ce qui est flou, bâcle tout ce qui est imprécis, et accélère la dette conceptuelle avec une efficacité terrifiante. Un cycle incomplet sans IA produit des logiciels imparfaits. Un cycle incomplet avec IA produit des logiciels imparfaits à une vitesse industrielle. Le SDLC réconcilié n'est pas archaïque. Il est indispensable — et le nier expose l'organisation à des risques que ni la vélocité de livraison ni la puissance des modèles ne pourront compenser.

Des concepts éprouvés bien avant nous

Il est une perspective que ce livre n'aurait pas rendu service à vous laisser ignorer. Si vous prenez le temps d'apprendre du passé — vraiment du passé, pas du passé de dix ou vingt ans, pas du passé des premières lignes de code COBOL ou des premiers manifestes Agile, mais du passé profond, du passé long, du passé qui précède les ordinateurs, la révolution industrielle, et peut-être même les civilisations dont nous avons conservé les archives — vous remarquerez quelque chose d'étonnant, et peut-être d'un peu troublant : les concepts sur lesquels ToTAL se fonde existaient bien avant l'industrie du logiciel. Bien avant l'informatique. Bien avant tout ce que nous appelons « moderne ».

Commencez par le concept le plus fondamental de ce livre : le cycle de vie. « Life cycle » n'est pas un concept informatique. Ce n'est pas une invention des années 1960. C'est un concept universel, inscrit dans la structure même de la nature — dans le cycle des saisons, dans le cycle de croissance des plantes, dans le cycle de vie des organismes, dans les rythmes astronomiques que les civilisations les plus anciennes observaient avec une précision qui nous confond encore. Les philosophies qui ont précédé notre ère technologique — grecques, chinoises, mésoaméricaines, égyptiennes — avaient toutes, sous des formes diverses et avec des vocabulaires qui nous semblent étrangers, cette compréhension profonde que rien ne se fait en ligne droite, que tout ce qui dure traverse des phases, et que la sagesse consiste non pas à accélérer ou à court-circuiter ces phases, mais à les comprendre assez profondément pour travailler avec elles plutôt que contre elles.

Et puis il y a le nom lui-même. **ToTAL** vient du monde Toltèque — une civilisation mésoaméricaine avancée dont nous connaissons à peine les contours aujourd'hui, et

dont les savoirs, transmis en grande partie oralement ou gravés dans des formes que nous n'avons pas encore pleinement déchiffrées, touchaient à des questions que notre époque redécouvre avec l'enthousiasme un peu naïf de celui qui croit inventer ce qu'il retrouve. Peut-être même avant les Toltèques, dans des savoirs perdus que nous n'avons pas encore redécouverts, dans des traditions de maîtrise et de cycle que le temps a ensevelies mais pas effacées. Une chose est certaine : le concept de cycle complet, de réconciliation des phases opposées, de maîtrise progressive acquise dans la répétition consciente et non dans la précipitation inconsciente — tout cela existait bien avant les années de production logicielle, bien avant l'IA, bien avant les ordinateurs, bien avant les langages de programmation.

Il y a dans l'histoire humaine ce phénomène récurrent, presque comique dans sa régularité si ses conséquences n'étaient pas si sérieuses : on oublie, on fragmente, on complique, on invente des solutions de plus en plus élaborées à des problèmes de plus en plus artificiels — et puis un jour, quelqu'un revient au gros bon sens, quelqu'un pose la question simple au milieu de la sophistication ambiante, quelqu'un dit « mais attendez, n'avons-nous pas toujours su que... ? » — et on dit « eurêka » en redécouvrant ce qui avait toujours fonctionné, ce qui avait toujours été là, ce qui n'avait jamais vraiment disparu mais que la frénésie du progrès avait recouvert d'une couche de bruit tellement épaisse qu'il était devenu inaudible. C'est exactement ce qui s'est passé avec le SDLC. Soixante ans de fragmentation. Soixante ans de guerres entre Waterfall et Agile, entre Dev et Ops, entre vitesse et rigueur, entre product et delivery, entre qualité et vélocité — comme si ces paires n'avaient jamais été des paires, mais des ennemis. Et puis : la réconciliation. Pas une invention. Une redécouverte. Le retour tranquille à ce qui avait toujours été là, sous le bruit, patient et complet.

> *Un jour revient, et l'évidence se pose comme une aube tranquille sur ce qui était là depuis toujours. C'est cela, ToTAL. Pas une révolution. Une reconnaissance.*

ToTAL est bien plus que le SDLC

Ce livre vous a raconté l'histoire du SDLC et la place que ToTAL y occupe. Il vous a conduit de la Cascade jusqu'au cycle réconcilié, à travers dix étapes qui sont autant de leçons sur la façon dont les organisations apprennent — lentement, douloureusement, souvent en oubliant avant de pouvoir aller plus loin. Il vous a montré comment ToTAL nomme ce cycle, le structure, lui rend sa cohérence. Mais ToTAL est bien plus que son SDLC réconcilié. Il est bien plus que ses six phases, ses trois sections ModDevOps, ses deux hémisphères SPECIFY et DELIVERY. Il est bien plus qu'un cycle, même complet et réconcilié.

ToTAL se fonde sur d'autres piliers — des fondations tout aussi solides, tout aussi éprouvées, tout aussi surprenantes dans leur profondeur lorsqu'on les examine honnêtement, sans les lunettes de la convention ni les œillères du marché. Des fondations que les autres livres de cette collection vous feront découvrir, l'une après l'autre, à leur propre rythme et avec leur propre profondeur. Chacun de ces livres est autonome — vous pouvez les lire séparément, dans l'ordre qui vous convient. Mais ensemble, ils forment quelque chose que chaque livre pris seul ne peut vous donner : une vision complète, une discipline intégrée, une façon de réaliser des produits logiciels qui est à la fois rigoureuse et humaine, exigeante et accessible, universelle et contextuelle.

L'**Excellence** — pas comme un idéal lointain réservé aux équipes d'élite ou aux organisations qui ont le luxe de ne pas se préoccuper du budget et des délais, mais comme une discipline quotidienne, incarnée dans les micro-décisions de chaque jour, accessible à toute équipe qui choisit de la pratiquer avec constance. Le Livre 3 vous montrera ce que l'excellence exige réellement — et ce qu'elle rend possible.

Le **Centre d'Excellence** — pas comme une structure bureaucratique supplémentaire dans un organigramme déjà surchargé, pas comme une police de la qualité qui viendra inspecter vos pratiques et distribuer des notes, mais comme une conscience collective au service des équipes, un catalyseur d'apprentissage organisationnel, un service qui diffuse la qualité sans imposer la conformité. Le Livre 4 vous donnera les outils pour le créer, le structurer, le pérenniser — et surtout, pour éviter les erreurs que la majorité des CoE commit dans leurs premiers mois.

La **Qualimétrie** — pas comme une obsession des chiffres, pas comme une accumulation de tableaux de bord qui rassure les directions sans informer les équipes, mais comme l'art de voir ce qui est invisible à l'œil nu : la santé réelle d'un système, la trajectoire réelle d'une équipe, la valeur réelle de ce qui est livré. Le Livre 5 vous apprendra à mesurer ce qui compte, à interpréter ce que les métriques révèlent — et ce qu'elles cachent.

La **Qualité** — dans sa définition la plus complète, la plus honnête, la plus utile que l'industrie du logiciel ait jamais eu besoin d'entendre. Pas la qualité comme conformité aux spécifications. Pas la qualité comme absence de bugs. La qualité comme adéquation entre ce qui est livré et ce qui était vraiment attendu — par les utilisateurs, par l'organisation, par le contexte dans lequel le logiciel doit vivre. Le Livre 6 vous donnera cette définition, et tout ce qu'elle implique.

BEASTEST — un acronyme qui cache une philosophie de test qui va bien au-delà de vérifier que le code fonctionne, bien au-delà des pyramides de tests et des métriques de couverture, bien au-delà de ce que la communauté QA a l'habitude d'appeler « qualité ». BEASTEST est une approche de la vérification qui commence bien avant le premier test écrit et se termine bien après le dernier déploiement. Le Livre 7 vous révélera ce que ces sept lettres contiennent vraiment.

Les **Anti-patterns** — ces comportements que les équipes reproduisent sans le savoir, ces dynamiques organisationnelles qui s'installent insidieusement, ces décisions qui semblent raisonnables dans l'instant et catastrophiques dans la durée. Le Livre 8 les nommera un par un, avec leurs symptômes, leurs causes profondes et leurs remèdes — parce que nommer ce qui nuit est la première étape pour s'en libérer.

La **Méthodologie ToTAL** — comment tout s'assemble dans un cadre cohérent et adaptable, comment les pièces que les autres livres vous ont données s'articulent les unes avec les autres, comment une équipe de cinq et une organisation de cinq mille peuvent toutes deux pratiquer ToTAL sans contradiction. Le Livre 9 est la vue d'ensemble, le plan du territoire complet.

L'**Architecture** — comme acte de maîtrise, pas comme contrainte technique imposée d'en haut, pas comme documentation que personne ne lit, mais comme la décision structurante qui détermine ce que le système pourra et ne pourra pas faire, maintenant et dans dix ans. Le Livre 10 vous montrera comment penser l'architecture dans le cycle ToTAL — et pourquoi l'ignorer coûte toujours plus cher qu'on ne le prévoit.

Le **Leadership** — ce qui fait qu'une équipe devient plus que la somme de ses membres, ce qui distingue le chef qui dirige de celui qui développe, ce qui explique pourquoi certaines équipes atteignent l'excellence presque naturellement tandis que d'autres, pourtant composées de personnes tout aussi talentueuses, n'y arrivent jamais. Le Livre 11 vous donnera les clés de ce mystère.

Le **Sabotage** — les forces silencieuses qui détruisent l'excellence de l'intérieur, ces dynamiques que personne ne nomme parce que personne ne veut être celui qui pointe les problèmes, ces comportements systémiques qui résistent à tous les cadres et toutes les méthodes parce qu'ils n'opèrent pas au niveau des processus mais au niveau des croyances et des relations. Le Livre 12 les reconnaîtra — et vous apprendra à les reconnaître avant qu'elles ne vous reconnaissent.

L'**IA dans ToTAL** — comment l'intelligence artificielle s'intègre au cycle réconcilié sans le remplacer, sans le déformer, sans en faire un gadget de productivité dont l'accélération efface le sens. Le Livre 13 est le livre le plus urgent de la collection pour ceux qui travaillent aujourd'hui avec des outils d'IA générative — parce qu'il dit exactement ce que ces outils peuvent amplifier, ce qu'ils ne peuvent pas remplacer, et ce qu'ils rendront désastreux si le cycle n'est pas complet.

La **Confiance** — le fondement silencieux de tout ce qui précède. Sans confiance entre les membres de l'équipe, les pratiques d'excellence restent des gestes isolés. Sans confiance entre les équipes et les parties prenantes, les spécifications restent des contrats méfiants. Sans confiance entre l'organisation et ses utilisateurs, la livraison reste un acte de foi aveugle. Le Livre 14 posera la question que tous les autres livres supposent résolue — et vous montrera comment la résoudre vraiment.

Et enfin, les sept **Guides pratiques** — Rouge, Orange, Jaune, Vert, Bleu, Indigo, Violet — qui vous accompagneront dans la mise en œuvre concrète de ToTAL selon votre niveau de maturité actuel. Chaque Guide est conçu pour là où vous êtes, pas pour là où vous aimeriez être. Chaque Guide est une boussole opérationnelle, un compagnon de terrain, un pont entre la compréhension intellectuelle de ToTAL et la pratique quotidienne qui le fait vivre dans vos équipes.

ToTAL est à la fois mystérieux dans ses origines et profondément humain dans ses manifestations. Il vous attend dans les pages qui suivent — non pas pour vous dicter une méthode, mais pour vous donner les mots, les outils et les repères qui vous permettront de réaliser, avec maîtrise et avec sens, ce que vous avez toujours su faire au fond : livrer un logiciel qui compte vraiment.

> ***« ToTAL ne réinvente pas le SDLC. Il le reconnaît. Et dans cette reconnaissance, il ouvre une porte vers quelque chose de bien plus grand — une discipline de maîtrise qui existait avant nous, qui existera après nous, et qui n'attendait qu'un nom pour être enfin pratiquée pleinement. »***
>
> — Michel Bélanger · Septembre 2026

© 2026 Michel Bélanger. Tous droits réservés.
Aucune partie de ce livre ne peut être reproduite, transmise ou stockée dans un système de récupération, sous quelque forme ou par quelque moyen que ce soit — électronique, mécanique, photocopie, enregistrement ou autre — sans l'autorisation écrite préalable de l'auteur.
Collection ToTAL est une œuvre originale. ToTAL, SENSE, SPECIFY, PLAN, FORGE, DELIVERY, CARE, BEASTEST, DREAM-TEAM et ORIGINALIUM sont des marques de la collection ToTAL.
Imprimé et publié au Canada · Septembre 2026